Mentry

Programmer's guide

2026-10-26

Contents

index **3**
mentry **3**
mentryDateTime **28**
mentryFixedPoint **33**
mentryIPAddr **34**
mentryIPv6Addr **36**
mentryThemes **38**
mentryWidget **42**
wcbRef **52**
License **62**

index

Keywords: index

The Multi-Entry Widget Package Mentry 4.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

Mentry Programmer's Guide
The mentry::mentry Command
Multi-Entry Widgets for Date and Time
Multi-Entry Widgets for Real Numbers in Fixed-Point Format
Multi-Entry Widgets for IP Addresses
Multi-Entry Widgets for IPv6 Addresses
Commands Related to Tile Themes
Wcb Command Reference

mentry

Keywords: mentry

Mentry Programmer's Guide

For Mentry Version 4.5.1

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

Overview
- What Is Mentry?
- How to Get It?
- How to Install It?
- How to Use It?
- More on Mentry_tile

Examples
- A mentry Widget for Phone Numbers
- A mentry Widget for Ethernet Addresses

- A mentry Widget for Franking IDs
- Using mentry Widgets for Date and Time
- Tile-Based Demo Scripts

Overview

What Is Mentry?

Mentry is a library package for Tcl/Tk versions 8.4 or higher, written in pure Tcl/Tk code. It contains:

- the implementation of the multi-entry mega-widget mentry, including a general utility module for mega-widgets;
- procedures that facilitate the use of multi-entry widgets for displaying and editing date and time in a great variety of formats, real numbers in fixed-point format, as well as IP(v4) addresses and IPv6 addresses;
- three richly commented demo scripts containing the typical steps needed to create and handle a mentry widget for a particular purpose;
- two demo scripts that show how to use the date- and time-related procedures;
- tile-based counterparts of the above-mentioned demo scripts;
- this tutorial;
- reference pages in HTML format.

A multi-entry widget consists of any number of entry widgets separated by labels, all embedded in a frame. Appropriately chosen configuration options make this conglomerate look like one single entry containing preinserted text pieces having invariant positions within the widget. The initial width of an entry component also determines the maximal number of characters that can be inserted into it; when reaching this limit in an entry having the input focus, the latter is set automatically to the next enabled entry component. The same action is triggered by typing a character contained in the label following the current entry, if the latter is non-empty.

Within a mentry widget, the `Left`, `Right`, `Home`, `End`, and `BackSpace` keys work across entry boundaries, while `Control-Left` and `Control-Right` play the same role as `Tab` and `Shift-Tab` in the case of ordinary entries.

Some of the above features are implemented with the aid of the widget callback package Wcb, written in pure Tcl/Tk code as well. **The Mentry package requires version 3.1 or higher of Wcb**, whose download location is

https://www.nemethi.de

It is very easy to create a multi-entry widget. For example, the command

```
mentry::mentry .me -body {3 - 3 - 4}
```

will create a mentry widget consisting of two entries of width 3 and one of width 4, separated by "`-`" characters. With the command

```
foreach w [.me entries] {
    wcb::cbappend $w before insert wcb::checkStrForNum
}
```

you can make sure that the three entries will only accept numeric input, thus providing a

comfortable and safe user interface for editing 10-digit phone numbers.

How to Get It?

Mentry is available for free download from the same URL as Wcb. The distribution file is `mentry4.5.1.tar.gz` for UNIX and `mentry4_5_1.zip` for Windows. These files contain the same information, except for the additional carriage return character preceding the linefeed at the end of each line in the text files for Windows.

Mentry is also included in tklib, which has the address

https://core.tcl.tk/tklib

How to Install It?

Install the package as a subdirectory of one of the directories given by the `auto_path` variable. For example, you can install it as a subdirectory of the `lib` directory within your Tcl/Tk installation.

To install Mentry *on UNIX*, `cd` to the desired directory and unpack the distribution file `mentry4.5.1.tar.gz`:

```
gunzip -c mentry4.5.1.tar.gz | tar -xf -
```

On most UNIX systems this can be replaced with

```
tar -zxf mentry4.5.1.tar.gz
```

Both commands will create a directory named `mentry4.5.1`, with the subdirectories `demos`, `doc`, and `scripts`.

On Windows, use WinZip or some other program capable of unpacking the distribution file `mentry4_5_1.zip` into the directory `mentry4.5.1`, with the subdirectories `demos`, `doc`, and `scripts`.

The file `mentryThemes.tcl` in the `scripts` directory is only needed for applications using the Mentry_tile package (see next section).

Notice that in tklib the Mentry `demos` directory is replaced with the subdirectory `mentry` of the `examples` directory. Please take this into account when reading the examples below.

How to Use It?

The Mentry distribution provides two packages, called **Mentry** and **Mentry_tile**. The main difference between the two is that Mentry_tile enables the tile-based, theme-specific appearance of mentry widgets; this package requires tile 0.6 or higher. It is not possible to use both packages in one and the same application, because both are implemented in the same `mentry` namespace and provide identical commands.

To be able to access the commands and variables of the Mentry package, your scripts must contain one of the lines

```
package require mentry ?version?
package require Mentry ?version?
```

You can use either one of the two statements above because the file `mentry.tcl` contains both lines

```
package provide mentry ...
package provide Mentry ...
```

Likewise, to be able to access the commands and variables of the Mentry_tile package, your scripts must contain one of the lines

```
package require mentry_tile ?version?
package require Mentry_tile ?version?
```

Again, you can use either one of the two statements above because the file `mentry_tile.tcl` contains both lines

```
package provide mentry_tile ...
package provide Mentry_tile ...
```

You are free to remove one of the above lines from `mentry.tcl` and `mentry_tile.tcl`, respectively, if you want to prevent the corresponding packages from making themselves known under two different names each. Of course, by doing so you restrict the argument of `package require`to a single name per package.

Since the packages Mentry and Mentry_tile are implemented in the `mentry` namespace, you must either invoke the

```
namespace import mentry::pattern ?mentry::pattern ...?
```

command to import the *procedures* you need, or use qualified names like `mentry::mentry`. In the examples below we have chosen the latter approach.

To access Mentry *variables*, you *must* use qualified names. There are only three Mentry variables (and one more when using Mentry_tile) that are designed to be accessed outside the namespace `mentry`:

- The variable `mentry::version` holds the current version number of the Mentry package.
- The variable `mentry::library` holds the location of the Mentry installation directory.
- The read-only variable `mentry::usingTile` has the value `0` in the Mentry package and the value `1` in Mentry_tile.
- In Mentry_tile the array `mentry::themeDefaults` holds the theme-specific default values of a series of Mentry configuration options.

More on Mentry_tile

As mentioned above, a mentry widget consists of entry and label widgets, embedded in a frame. While in the Mentry package all of these components are Tk widgets, the Mentry_tile package uses both Tk frame, tile entry, and Tk label widgets. Due to several incompatibilities between Tk and tile, it is currently not possible to replace all Tk widgets making up a mentry with their tile counterparts. Actually, the entry components of a tile-based mentry are embedded into Tk frame widgets, which in turn, together with the labels, are packed into a specially constructed tile entry rather than a frame. This somewhat complicated layout is needed because in several themes it is not possible to draw flat, borderless tile enty widgets.

From the above it follows that **the Mentry_tile package will only work as expected if the Tk frame and label commands haven't been overridden by using `namespace import -force ttk::*` at global scope**. While earlier tile releases suggested using this command at

global scope for the really adventurous, in newer tile versions this is considered a Really Bad Idea, causing many things to break. Instead, **you should explicitly invoke ttk::frame, ttk::label, etc. whenever you want to use a tile widget**.

Another restriction to be taken into account is due to the fact that in earlier tile versions the `(ttk::)style theme use`command could only be used to set the current theme, but not to retrieve it. For this reason, if the Mentry_tile package cannot get the current theme with the aid of `ttk::style theme use`then it makes use of the variable `ttk::currentTheme` or `tile::currentTheme` (depending on the tile version), which is set by the `ttk::setTheme` or `tile::setTheme` command. From this it follows that **if the tile version being used doesn't support the ttk::style theme use command without an argument then the tile-based mentry widgets will only have the expected appearance if the platform-specific default theme is either left unchanged or replaced with another theme by invoking the library procedure ttk::setTheme or tile::setTheme, depending on the tile version**. (See also the `mentry::setTheme` command.)

After these cautions concerning the use of tile, the rest of this section describes the differences between the packages Mentry and Mentry_tile.

The Mentry_tile package checks whether the required Tk and tile versions are present, by executing the commands

```
package require Tk 8.4-
if {$::tk_version < 8.5 || [regexp {^8\.5a[1-5]$} $::tk_patchLevel]} {
    package require tile 0.6-
}
```

The second command above reflects the fact that, beginning with Tk 8.5a6, tile is integrated into the Tk core and therefore it should only be loaded explicitly when using an earlier Tk version.

Apart from this and the `_tile` suffix in the `package require`command, the only difference (from the programmer's point of view) between the packages Mentry and Mentry_tile is related to the supported configuration options: The following Tk (entry) widget options, present in the Mentry package, are not supported by Mentry_tile, because they are not available for tile (entry) widgets: `-borderwidth`, `-disabledbackground`, `-disabledforeground`, `-highlightbackground`, `-highlightcolor`, `-highlightthickness`, `-insertbackground`, `-insertborderwidth`, `-insertofftime`, `-insertontime`, `-insertwidth`, `-readonlybackground`, `-relief`, `-selectbackground`, `-selectborderwidth`, and `-selectforeground`.

Notice that the `-background` option doesn't work as expected if the current theme is `Arc`, `plastik`, `tileqt`, `vista`, or `xpnative`, because these themes silently ignore any attempt to change the background color of a tile entry widget.

Finally, take into account that, when using the `tileqt` theme, the version number of the `tile::theme::tileqt` package must be 0.4 or higher, and `tileqt` itself won't work with tile versions earlier than 0.7.

Examples

A mentry Widget for Phone Numbers

Let's resume the example mentioned in the Overview in a bit more systematical manner. First, we will write a procedure for creating a mentry widget that allows to display and edit 10-digit phone numbers and accepts any configuration options supported by the `mentry::mentry` command:

```
#-------------------------------------------------------------------------
    --------
# phoneNumberMentry
#
# Creates a new mentry widget win that allows to display and edit
    10-digit
# phone numbers.  Sets the type attribute of the widget to PhoneNumber
    and
# returns the name of the newly created widget.
#-------------------------------------------------------------------------
    --------
proc phoneNumberMentry {win args} {
    #
    # Create a mentry widget consisting of two entries of width 3 and
    one of
    # width 4, separated by "-" characters, and set its type to
    PhoneNumber
    #
    eval [list mentry::mentry $win] $args
    $win configure -body {3 - 3 - 4}
    $win attrib type PhoneNumber

    #
    # Allow only decimal digits in all entry components; use
    # wcb::cbappend (or wcb::cbprepend) instead of wcb::callback
    # in order to keep the wcb::checkEntryLen callback,
    # registered by mentry::mentry for all entry components
    #
    for {set n 0} {$n < 3} {incr n} {
        set w [$win entrypath $n]
        wcb::cbappend $w before insert wcb::checkStrForNum
        $win adjustentry $n "0123456789"
        bindtags $w [linsert [bindtags $w] 1 MentryPhoneNumber]
    }

    return $win
}
```

The first argument `win` is the name of the widget, and the keyword `args` represents a list of configuration options and their values, just like in the case of the standard Tk widgets. The value `{3 - 3 - 4}`of the `-body` option specifies that the mentry should consist of two entries of width 3 and one of width 4, separated by labels displaying the `"-"` character.

Each mentry widget may have any number of private **attributes**, which can be set and retrieved with the aid of the `attrib` subcommand of the Tcl procedure corresponding to the widget. We use this subcommand to define the `type` attribute of the newly created widget and set it to the value `"PhoneNumber"`. Although this is not strictly necessary, it will enable us to distinguish a phone number mentry from other multi-entry widgets.

The `mentry::mentry` command registers the `wcb::checkEntryLen` callback with each entry component of the mentry widget to restrict the number of characters that can be inserted into it to the initial width specified in the `-body` option. Besides this constraint, we want our entries to accept only decimal digits, therefore we use the `wcb::cbappend` command to *add* the procedure `wcb::checkStrForNum` to the callback list of each entry component. By invoking `wcb::callback` instead of `wcb::cbappend` (or `wcb::cbprepend`), we would *replace* the callback list with the one consisting of the single element `wcb::checkStrForNum`.

Now we know that each entry component of the mentry widget will only accept a limited number of decimal digits. But are the widths of the entries large enough to hold the maximal number of 3 or 4 decimal digits, respectively? In the case of a fixed-width font the answer is definitely "yes", and the same holds true for most proportionally-spaced fonts. There are, however, fonts in which not all decimal digits have the same width. For this reason, we invoke the `adjustentry` subcommand for each entry component, passing to it as last argument a string consisting of the allowed characters, which in this example are the decimal digits. This subcommand will increase the entry widget's width if needed, to make it just large enough for texts of the component-specific maximal length, consisting of characters specified by that string.

Our second procedure outputs a phone number to a mentry widget having a `type` attribute value of `"PhoneNumber"`:

```
#-------------------------------------------------------------------------
    --------
# putPhoneNumber
#
# Outputs the phone number num to the mentry widget win of type
    PhoneNumber.
# The phone number must be a string of length 10, consisting of
    decimal digits.
#-------------------------------------------------------------------------
    --------
proc putPhoneNumber {num win} {
    #
    # Check the syntax of num
    #
    if {[string length $num] != 10 || ![regexp {^[0-9]*$} $num]} {
        return -code error "expected 10 decimal digits but got
    \"$num\""
    }

    #
    # Check the widget and display the properly formatted phone number
    #
```

```
    checkIfPhoneNumberMentry $win
    $win put 0 [string range $num 0 2] [string range $num 3 5] \
               [string range $num 6 9]
}
```

We use the `put` subcommand of the Tcl procedure corresponding to the mentry widget to display the three substrings of the given phone number in the corresponding entries, starting with the entry whose index is specified as the first argument following the word `put`.

Next, we need a procedure that returns the phone number contained in a mentry widget having a `type` attribute value of `"PhoneNumber"`:

```
#--------------------------------------------------------------------------
    --------
# getPhoneNumber
#
# Returns the phone number contained in the mentry widget win of type
# PhoneNumber.
#--------------------------------------------------------------------------
    --------
proc getPhoneNumber win {
    #
    # Check the widget
    #
    checkIfPhoneNumberMentry $win

    #
    # Generate an error if any entry component is empty or incomplete
    #
    for {set n 0} {$n < 3} {incr n} {
        if {[$win isempty $n]} {
            focus [$win entrypath $n]
            return -code error EMPTY
        }
        if {![$win isfull $n]} {
            focus [$win entrypath $n]
            return -code error INCOMPL
        }
    }

    #
    # Return the phone number built from the
    # values contained in the entry components
    #
    $win getarray strs
    return $strs(0)$strs(1)$strs(2)
}
```

The procedure runs over the indices of the entry components of the given mentry widget and invokes the `isempty` and `isfull` subcommands of the Tcl command corresponding to the given mentry widget. If one of the entries is found to be empty or incomplete, the procedure gets its path name by calling the `entrypath` subcommand, sets the focus to that entry, raises an error, and returns the value `"EMPTY"` or `"INCOMPL"`, respectively. The application invoking this procedure should then display an appropriate error message corresponding to the return value.

Notice that the number `3` in the `for` loop above is nothing else than `[$win entrycount]`. Also, it would be sufficient to check whether all entry components are full, because an empty entry is at the same time incomplete. The preliminary check whether an entry is empty is just made for the user's convenience.

To build the phone number from the values contained in the entry components, we use a temporary array variable and invoke the `getarray` subcommand, which copies the contents of the entries to the corresponding array elements.

The last two procedures presented above contain an invocation of the command `checkIfPhoneNumberMentry`, which is implemented as folows:

```
#-------------------------------------------------------------------------
    --------
# checkIfPhoneNumberMentry
#
# Generates an error if win is not a mentry widget of type PhoneNumber.
#-------------------------------------------------------------------------
    --------
proc checkIfPhoneNumberMentry win {
    if {![winfo exists $win]} {
        return -code error "bad window path name \"$win\""
    }

    if {[winfo class $win] ne "Mentry" ||
        [$win attrib type] ne "PhoneNumber"} {
        return -code error \
               "window \"$win\" is not a mentry widget for phone
    numbers"
    }
}
```

This procedure retrieves the value of the `type` attribute of its argument to check whether the latter denotes a mentry widget for phone numbers (recall that this attribute was set to the value `"PhoneNumber"` in the procedure `phoneNumberMentry`).

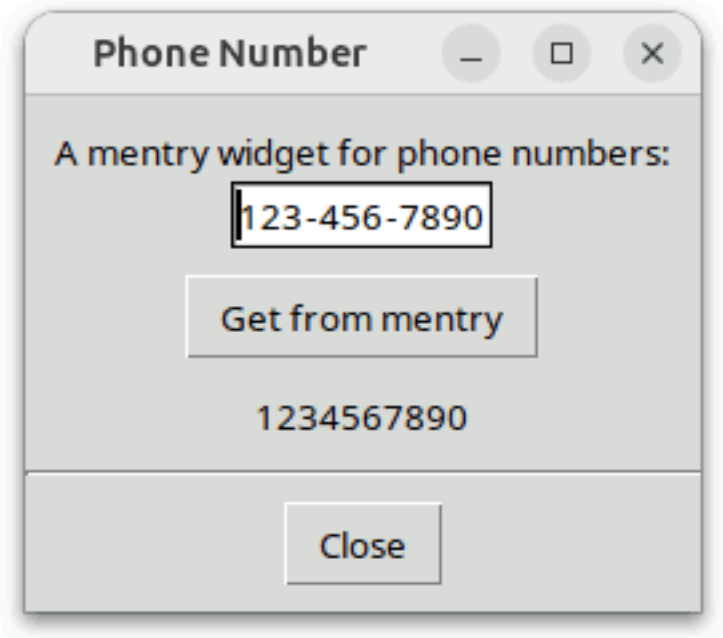
Finally, recall that in the procedure `phoneNumberMentry`, we inserted the binding tag `MentryPhoneNumber` into the list of binding tags of the mentry's entry components, just after their path names. The binding script for this tag shown in the code below handles `<<Paste>>` (`<Control-v>` or `<Command-v>`) events in these entry widgets by pasting the current contents of the clipboard into the mentry, provided that it is a valid 10-digit phone number:

```
bind MentryPhoneNumber <<Paste>> { pastePhoneNumber %W }

#-------------------------------------------------------------------------
    --------
# pastePhoneNumber
#
# Handles <<Paste>> events in the entry component w of a mentry widget
    for
# 10-digit phone numbers by pasting the current contents of the
    clipboard into
# the mentry if it is a valid 10-digit phone number.
#-------------------------------------------------------------------------
    --------
proc pastePhoneNumber w {
    set res [catch {::tk::GetSelection $w CLIPBOARD} num]
    if {$res == 0} {
        set win [winfo parent $w]
        catch { putPhoneNumber $num $win }
    }

    return -code break ""
}
```

The five procedures discussed above are implemented in the file `phonenumber.tcl`, contained in the `demos` directory. This script also puts them together to build a small application displaying the following figure:



Phone Number

Here is the relevant code fragment:

```
package require mentry

set title "Phone Number"
wm title . $title

#
# Add some entries to the Tk option database
#
source [file join [file dirname [info script]] option.tcl]

. . .

#
# Frame .f with a mentry displaying a phone number
#
frame .f
label .f.l -text "A mentry widget for phone numbers:"
phoneNumberMentry .f.me
pack .f.l .f.me

#
# Message strings corresponding to the values
# returned by getPhoneNumber on failure
#
array set msgs {
    EMPTY       "Field value missing"
    INCOMPL     "Incomplete field value"
}

#
# Button .get invoking the procedure getPhoneNumber
#
button .get -text "Get from mentry" -command {
    if {[catch {
        set num ""
        set num [getPhoneNumber .f.me]
    } result] != 0} {
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
}

#
# Label .num displaying the result of getPhoneNumber
#
```

```
label .num -textvariable num

. . .

putPhoneNumber 1234567890 .f.me
focus [.f.me entrypath 0]
```

A mentry Widget for Ethernet Addresses

Ethernet addresses are usuallly written in the form `"XX:XX:XX:XX:XX:XX"`, where each `"X"` is a hexadecimal digit. The file `ethernetaddr.tcl` in the `demos` directory contains the steps needed to create and use a multi-entry widget for displaying and editing Ethernet addresses. It implements the procedures `ethernetAddrMentry`, `putEthernetAddr`, and `getEthernetAddr`; the last two invoke the helper procedure `checkIfEthernetAddrMentry`, while the first one is implemented as follows:

```
#-------------------------------------------------------------------------
    --------
# ethernetAddrMentry
#
# Creates a new mentry widget win that allows to display and edit
    Ethernet
# addresses.  Sets the type attribute of the widget to EthernetAddr
    and returns
# the name of the newly created widget.
#-------------------------------------------------------------------------
    --------
proc ethernetAddrMentry {win args} {
    #
    # Create a mentry widget consisting of 6 entries of width
    # 2, separated by colons, and set its type to EthernetAddr
    #
    eval [list mentry::mentry $win] $args
    $win configure -body {2 : 2 : 2 : 2 : 2 : 2}
    $win attrib type EthernetAddr

    #
    # Install automatic uppercase conversion and allow only hexadecimal
    # digits in all entry components; use wcb::cbappend (or
    wcb::cbprepend)
    # instead of wcb::callback in order to keep the wcb::checkEntryLen
    # callback, registered by mentry::mentry for all entry components
    #
    for {set n 0} {$n < 6} {incr n} {
        set w [$win entrypath $n]
        wcb::cbappend $w before insert wcb::convStrToUpper \
                      {wcb::checkStrForRegExp {^[0-9A-F]*$}}
```

```
        $win adjustentry $n "0123456789ABCDEF"
        bindtags $w [linsert [bindtags $w] 1 MentryEthernetAddr]
    }

    return $win
}
```

Notice again the invocation of the `adjustentry` subcommand of the Tcl command associated with the mentry widget, for each of its entry components. This is necessary, because in the case of a proportionally-spaced font the characters `A - F` need more room than the digits `0 - 9` (and it is not even guaranteed that the latters have the same width).

The procedure `putEthernetAddr` expects as its first argument a string of the form `"XX:XX:XX:XX:XX:XX"`, where each `"XX"` must be a hexadecimal string in the range `0 - 255`:

```
#-------------------------------------------------------------------------
    --------
# putEthernetAddr
#
# Outputs the Ethernet address addr to the mentry widget win of type
# EthernetAddr.  The address must be a string of the form
    XX:XX:XX:XX:XX:XX,
# where each XX must be a hexadecimal string in the range 0 - 255.
    Leading
# zeros are allowed (but not required), hence the components may have
    more (but
# also less) than two characters; the procedure displays them with
    exactly two
# digits.
#-------------------------------------------------------------------------
    --------
proc putEthernetAddr {addr win} {
    set errorMsg "expected an Ethernet address but got \"$addr\""

    #
    # Check the syntax of addr
    #
    set lst [split $addr :]
    if {[llength $lst] != 6} {
        return -code error $errorMsg
    }

    #
    # Try to convert the 6 components of addr to hexadecimal
    # strings and check whether they are in the range 0 - 255
    #
```

```
    for {set n 0} {$n < 6} {incr n} {
        set val 0x[lindex $lst $n]
        if {[catch {format "%02X" $val} str$n] != 0 || $val < 0 ||
    $val > 255} {
            return -code error $errorMsg
        }
    }

    #
    # Check the widget and display the properly formatted Ethernet
    address
    #
    checkIfEthernetAddrMentry $win
    $win put 0 $str0 $str1 $str2 $str3 $str4 $str5
}
```

The procedure `getEthernetAddr` raises an error if any entry component of the given mentry widget is empty. It accepts also entry strings of length one, but in the return value all components will have exactly two digits:

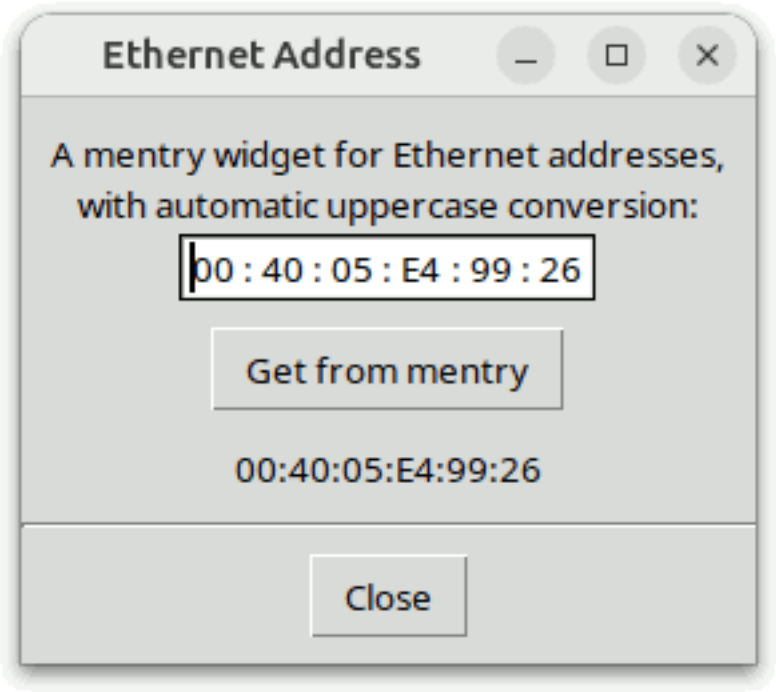
```
#-------------------------------------------------------------------------
    --------
# getEthernetAddr
#
# Returns the Ethernet address contained in the mentry widget win of
    type
# EthernetAddr.
#-------------------------------------------------------------------------
    --------
proc getEthernetAddr win {
    #
    # Check the widget
    #
    checkIfEthernetAddrMentry $win

    #
    # Generate an error if any entry component is empty
    #
    for {set n 0} {$n < 6} {incr n} {
        if {[$win isempty $n]} {
            focus [$win entrypath $n]
            return -code error EMPTY
        }
    }

    #
    # Return the properly formatted Ethernet address built
```

```
    # from the values contained in the entry components
    #
    $win getarray strs
    return [format "%02X:%02X:%02X:%02X:%02X:%02X" \
            0x$strs(0) 0x$strs(1) 0x$strs(2) 0x$strs(3) 0x$strs(4)
    0x$strs(5)]
}
```

We will not show the rest of the code here, because it is very similar to the one presented in the preceding section. The mentry widget for Ethernet addresses looks like in the following screenshot:



Ethernet Address

A mentry Widget for Franking IDs

The German postal service (Deutsche Post) uses 20-character Franking IDs for tracking and sorting mail pieces. Franking IDs are usually written as strings of the form `"XX XXXX XXXX XX XXXX XXXX"`, where each `"X"` is an uppercase hexadecimal digit and the last one is a CRC checksum of type CCITT (CRC-4). The file `frankingid.tcl` in the `demos` directory contains the steps needed to create and use a multi-entry widget for displaying and editing Franking IDs. It implements the main procedures `frankingIdMentry`, `putFrankingId`, and `getFrankingId`, along with the helper ones `checkIfFrankingIdMentry`, `crc4`, and `pasteFrankingId`.

The procedure `frankingIdMentry`, which creates a mentry widget for Franking IDs, is implemented as follows:

```
#-------------------------------------------------------------------------
    --------
# frankingIdMentry
#
# Creates a new mentry widget win that allows to display and edit
    20-character
# Franking IDs.  Sets the type attribute of the widget to FrankingId
    and
# returns the name of the newly created widget.
#-------------------------------------------------------------------------
    --------
proc frankingIdMentry {win args} {
    #
    # Create a mentry widget consisting of 6 entries of widths 2, 4,
    4, 2, 4,
    # and 4, separated by " " characters, and set its type to
    FrankingId
    #
    eval [list mentry::mentry $win] $args
    $win configure -body {2 " " 4 " " 4 " " 2 " " 4 " " 4}
    $win attrib type FrankingId

    #
    # Install automatic uppercase conversion and allow only hexadecimal
    # digits in all entry components; use wcb::cbappend (or
    wcb::cbprepend)
    # instead of wcb::callback in order to keep the wcb::checkEntryLen
    # callback, registered by mentry::mentry for all entry components
    #
    for {set n 0} {$n < 6} {incr n} {
        set w [$win entrypath $n]
        wcb::cbappend $w before insert wcb::convStrToUpper \
                      {wcb::checkStrForRegExp {^[0-9A-F]*$}}
        $win adjustentry $n "0123456789ABCDEF"
        bindtags $w [linsert [bindtags $w] 1 MentryFrankingId]
    }

    return $win
}
```

The procedure `putFrankingId` accepts as its first argument both an unformatted (20-character) Franking ID and one of the form `"XX XXXX XXXX XX XXXX XXXX"`:

```
#-------------------------------------------------------------------------
    --------
# putFrankingId
#
```

```
# Outputs the Franking ID id to the mentry widget win of type
    FrankingId.  The
# Franking ID must be either a string consisting of 20 uppercase
    hexadecimal
# digits, or a string of the form "XX XXXX XXXX XX XXXX XXXX", where
    each X is
# an uppercase hexadecimal digit.
#-------------------------------------------------------------------
    --------
proc putFrankingId {id win} {
    #
    # Check the syntax of num
    #
    set expr1 {^[0-9A-F]{20}$}
    set expr2 {^[0-9A-F]{2} [0-9A-F]{4} [0-9A-F]{4} [0-9A-F]{2}
    [0-9A-F]{4}\
                [0-9A-F]{4}$}
    if {[regexp $expr1 $id]} {
        set isFormatted 0
    } elseif {[regexp $expr2 $id]} {
        set isFormatted 1
    } else {
        return -code error "expected a Franking ID but got \"$id\""
    }

    #
    # Check the widget and display the properly formatted Franking ID
    #
    checkIfFrankingIdMentry $win
    if {$isFormatted} {
        foreach {str0 str1 str2 str3 str4 str5} [split $id " "] {}
        $win put 0 $str0 $str1 $str2 $str3 $str4 $str5
    } else {
        $win put 0 [string range $id 0 1] [string range $id 2 5] \
                   [string range $id 6 9] [string range $id 10 11] \
                   [string range $id 12 15] [string range $id 16 19]
    }
}
```

Likewise, the Franking ID returned by the procedure `getFrankingId` can be a formatted or unformatted one, depending on the boolean value of its optional argument:

```
#-------------------------------------------------------------------
    --------
# getFrankingId
#
# Returns the Franking ID contained in the mentry widget win of type
```

```
# FrankingId, per default formatted as a string of the form
# "XX XXXX XXXX XX XXXX XXXX", where each X is an uppercase
    hexadecimal digit.
#-------------------------------------------------------------------------
    --------
proc getFrankingId {win {formatted 1}} {
    #
    # Check the widget
    #
    checkIfFrankingIdMentry $win

    #
    # Generate an error if any entry component is empty or incomplete.
    # "Incomplete" means in case of the last entry that it has less
    than
    # 3 characters, and in case of the other entries that it is not
    full.
    #
    for {set n 0} {$n < 6} {incr n} {
        if {[$win isempty $n]} {
            focus [$win entrypath $n]
            return -code error EMPTY
        }

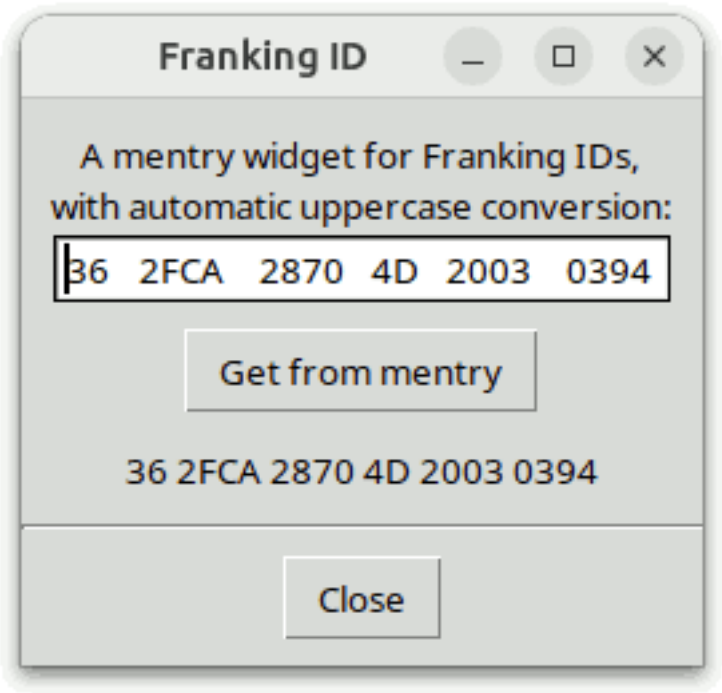
        if {$n == 5} {
            set w [$win entrypath $n]
            if {[string length [$w get]] < 3} {
                focus $w
                return -code error INCOMPL
            }
        } elseif {![$win isfull $n]} {
            focus [$win entrypath $n]
            return -code error INCOMPL
        }
    }

    #
    # Return the Franking ID built from the values contained in
    # the entry components and extended by the checksum if needed
    #
    set id [$win getstring]
    if {[string length $id] == 24} {  ;# i.e., last entry contains 3
    characters
        append id [crc4 $id]
    } elseif {[string index $id end] ne [crc4 [string range $id 0
    end-1]]} {
```

```
        focus $w
        return -code error BAD_CRC
    }
    return [expr {$formatted ? $id : [string map {" " ""} $id]}]
}
```

As seen in the code above, if the last entry contains 3 characters only then the procedure computes the checksum and appends it to the result. Otherwise it checks whether the last character equals the checksum built from the first 24 characters (of which only 19 are significant), and raises an error if this is not the case. (Note that the checksum algorithm ignores the space characters.)

Again, we will not show the rest of the code here. The mentry widget for Franking IDs looks like in the following screenshot:



Franking ID

Using mentry Widgets for Date and Time

Multi-entry widgets can be used to display and edit date and time in a great variety of formats. The Mentry package contains ready-to-use commands for this purpose:

- The procedure `mentry::dateMentry` creates a new mentry widget for displaying and editing a date according to the format passed to the command as a three-character string consisting of the field descriptor characters `"d"`, `"m"`, and `"y"` or `"Y"`, known from the `clock format` command. Another argument expected by the procedure is the string (usually a single character) to be displayed in the labels separating the three components of the date.
- Similarly, the procedure `mentry::timeMentry` creates a new mentry widget for displaying and editing a time according to the format passed to the command as a two- or three-character string consisting of the following field descriptor characters of the `clock format`command: `"H"` or `"I"`, followed by `"M"`, and optionally the letter `"S"`. An `"H"` as first character specifies the time format `"%H:%M"` or `"%H:%M:%S"` (i.e., with the hour between `0` and `23`),

while the letter `"I"` stands for `"%I:%M %p"`or `"%I:%M:%S %p"`(i.e., with AM/PM indicator). The procedure expects the separator string (which is usually the `":"` character) as another command-line argument.
- Finally, the procedure `mentry::dateTimeentry` creates a new mentry widget for displaying and editing a date & time according to the format passed to the command as a 5- or 6-character string, with the first three characters consisting of the letters `"d"`, `"m"`, and `"y"` or `"Y"`, followed by two or three further field descriptor characters of the `clock format` command, which must be: `"H"` or `"I"`, then `"M"`, and optionally the letter `"S"`. The procedure expects two more arguments, specifying the separators to be used in the date and time parts of the mentry widget, respectively.

Before describing the other date- and time-related commands provided by the Mentry package, let's see how the first two of the above are invoked in the file `datetime1.tcl`, located in the `demos` directory:

```
package require mentry

set title "Date & Time"
wm title . $title

#
# Add some entries to the Tk option database
#
source [file join [file dirname [info script]] option.tcl]

#
# Date and time formats supported by this demo
# script and the corresponding field separators
#
array set dateFmts {0 mdy  1 dmy  2 Ymd}
array set dateSeps {0 /    1 .    2 -  }
array set timeFmts {0 IMS  1 HMS}
array set timeSeps {0 :    1 :  }

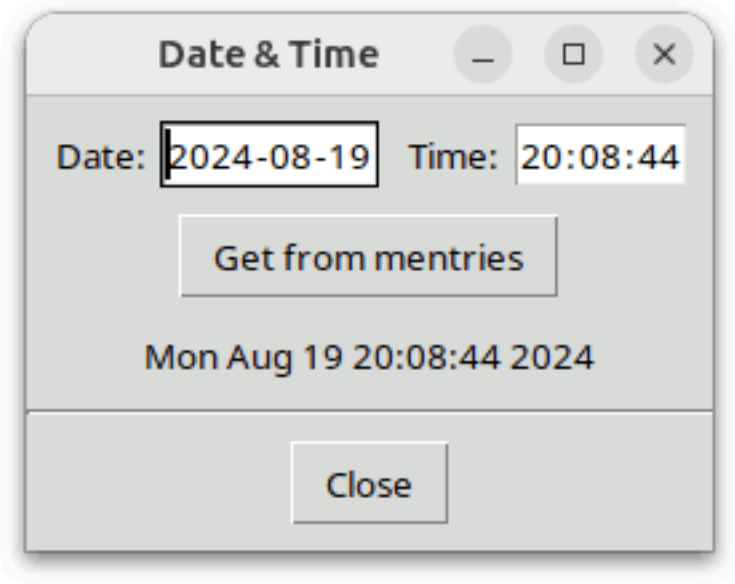
#
# Choose the date & time formats; don't use the %p field descriptor
# for displaying the AM/PM indicator, because it doesn't
# work on UNIX if Tcl/Tk is used in a non-default locale
#
wm withdraw .
set clockVal [clock seconds]
if {[clock format $clockVal -format "%H"] < 12} {
    set meridian AM
} else {
    set meridian PM
}
set dateIdx [tk_dialog .choice $title "Please choose a date format" {}
```

```
    -1 \
                        [clock format $clockVal -format "%m/%d/%y"] \
                        [clock format $clockVal -format "%d.%m.%y"] \
                        [clock format $clockVal -format "%Y-%m-%d"]]
set timeIdx [tk_dialog .choice $title "Please choose a time format" {}
    -1 \
                        [clock format $clockVal -format "%I:%M:%S
    $meridian"] \
                        [clock format $clockVal -format "%H:%M:%S"]]
wm deiconify .

#
# Frame .f with mentries displaying the date & time
#
frame .f
label .f.lDate -text "Date: "
mentry::dateMentry .f.date $dateFmts($dateIdx) $dateSeps($dateIdx) \
                   -justify center
frame .f.gap -width 10
label .f.lTime -text "Time: "
mentry::timeMentry .f.time $timeFmts($timeIdx) $timeSeps($timeIdx) \
                   -justify center
pack .f.lDate .f.date .f.gap .f.lTime .f.time -side left
```

Before displaying the main window, the script lets the user choose one out of three date and one out of two time formats. The corresponding command-line arguments passed to `mentry::dateMentry` and `mentry::timeMentry` are taken from the arrays `dateFmts`, `dateSeps`, `timeFmts`, and `timeSeps`.

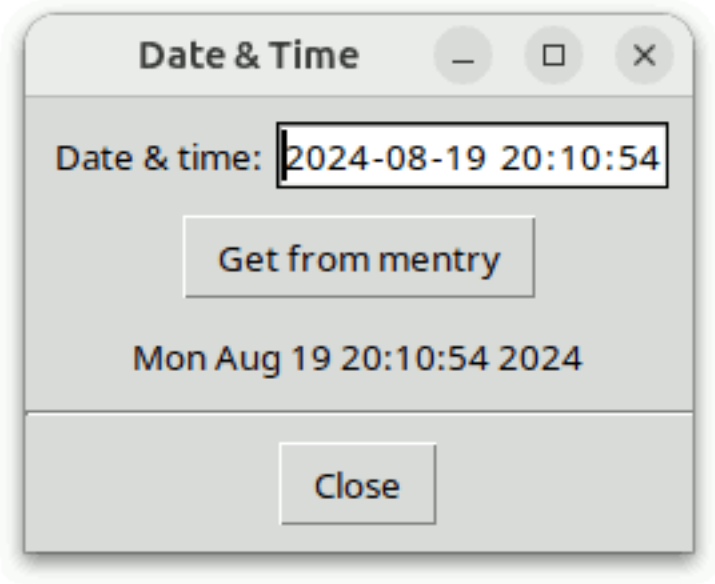
The following figure corresponds to the choices `dateIdx = 2`and `timeIdx = 1`:



Date & Time

The demo script `datetime2.tcl` displays both the date and time in the same mentry widget, with the aid of the third command described above:

```
#
# Frame .f with a mentry displaying the date & time
#
frame .f
label .f.l -text "Date & time: "
mentry::dateTimeMentry .f.me $dateFmts($dateIdx)$timeFmts($timeIdx) \
                       $dateSeps($dateIdx) $timeSeps($timeIdx) \
                       -justify center
pack .f.l .f.me
```



Date & Time

The Mentry package exports two further commands for date, time, and date & time mentries:

- The procedure `mentry::putClockVal` outputs the date, time, or date & time corresponding to an integer clock value specified as its first argument to a date, time, or date & time mentry widget, passed to it as the second parameter. Like the`clock format`command, the procedure accepts the optional argument pair `-gmt boolean`.
- The procedure `mentry::getClockVal` returns the clock value corresponding to the date, time, or date & time contained in the date, time, or date & time mentry specified as its first argument. Like the`clock scan`command, the procedure accepts the optional argument pairs `-base clockValue`and `-gmt boolean`. On failure, the procedure sets the focus to the first erronous entry component, generates an error, and returns one of the values contained in the following code fragment taken from the scripts `datetime1.tcl` and `datetime2.tcl`:

  The string `"EMPTY"` is returned if any entry component (except the one containing the seconds) was found to be empty. The value `"BAD"` means a day, month, or hour value of `0` (the hour must not be `0` if the AM/PM indicator is present). The string `"BAD_DATE"` is returned when the <year, month, day> triple is invalid (note that the procedure is aware of leap years). Finally, even if this triple is valid, the conversion (made with the aid of the `clock scan`command) can fail because of an unsupported year value (e.g., between `38` and `70`); in

this case the string `"BAD_YEAR"` is returned.

```
#
# Message strings corresponding to the values
# returned by mentry::getClockVal on failure
#
array set msgs {
    EMPTY       "Field value missing"
    BAD         "Invalid field value"
    BAD_DATE    "Invalid date"
    BAD_YEAR    "Unsupported year"
}
```

The demo script `datetime1.tcl` invokes the last two commands as follows:

```
#
# Button .get invoking the procedure mentry::getClockVal
#
button .get -text "Get from mentries" -command {
    if {[catch {
        set dateTime ""
        set base [mentry::getClockVal .f.date]
        set clockVal [mentry::getClockVal .f.time -base $base]
        set dateTime [clock format $clockVal -format "%c"]
    } result] != 0} {
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
}

#
# Label .dateTime displaying the result of mentry::getClockVal
#
label .dateTime -textvariable dateTime

. . .

set clockVal [clock seconds]
mentry::putClockVal $clockVal .f.date
mentry::putClockVal $clockVal .f.time
focus [.f.date entrypath 0]
```

To obtain the clock value from the mentry widgets `.f.date` and `.f.time`, we first pass the name of the date mentry to the command `mentry::getClockVal` and then use the result as the value of the `-base` option when passing the name of the time mentry to the same procedure.

The demo script `datetime2.tcl` is simpler:

```
#
# Button .get invoking the procedure mentry::getClockVal
#
button .get -text "Get from mentry" -command {
    if {[catch {
        set dateTime ""
        set clockVal [mentry::getClockVal .f.me]
        set dateTime [clock format $clockVal -format "%c"]
    } result] != 0} {
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
}

#
# Label .dateTime displaying the result of mentry::getClockVal
#
label .dateTime -textvariable dateTime

. . .

set clockVal [clock seconds]
mentry::putClockVal $clockVal .f.me
focus [.f.me entrypath 0]
```

Tile-Based Demo Scripts

The Mentry distribution contains also tile-based counterparts of the demo scripts discussed above. As described in the More on Mentry_tile section of this tutorial, it is quite easy to port an application using the Mentry package to one based on Mentry_tile. For example, let's see how to transform the demo script `phonenumber.tcl` into a tile-based one, called `phonenumber_tile.tcl`. The changes are shown below in red color:

```
package require mentry_tile

set title "Phone Number"
wm title . $title

#
# Add some entries to the Tk option database
#
source [file join [file dirname [info script]] option_tile.tcl]

. . .

proc pastePhoneNumber w {
```

```
    . . .

    if {$res == 0} {
        set win [winfo parent [winfo parent $w]]
        . . .
    }
}

. . .

#
# Improve the window's appearance by using a tile
# frame as a container for the other widgets
#
ttk::frame .base

#
# Frame .base.f with a mentry displaying a phone number
#
ttk::frame .base.f
ttk::label .base.f.l -text "A mentry widget for phone numbers:"
phoneNumberMentry .base.f.me
pack .base.f.l .base.f.me

#
# Message strings corresponding to the values
# returned by getPhoneNumber on failure
#
array set msgs {
    EMPTY       "Field value missing"
    INCOMPL     "Incomplete field value"
}

#
# Button .base.get invoking the procedure getPhoneNumber
#
ttk::button .base.get -text "Get from mentry" -command {
    if {[catch {
        set num ""
        set num [getPhoneNumber .base.f.me]
    } result] != 0} {
        bell
        tk_messageBox -icon error -message $msgs($result) \
                      -title $title -type ok
    }
```

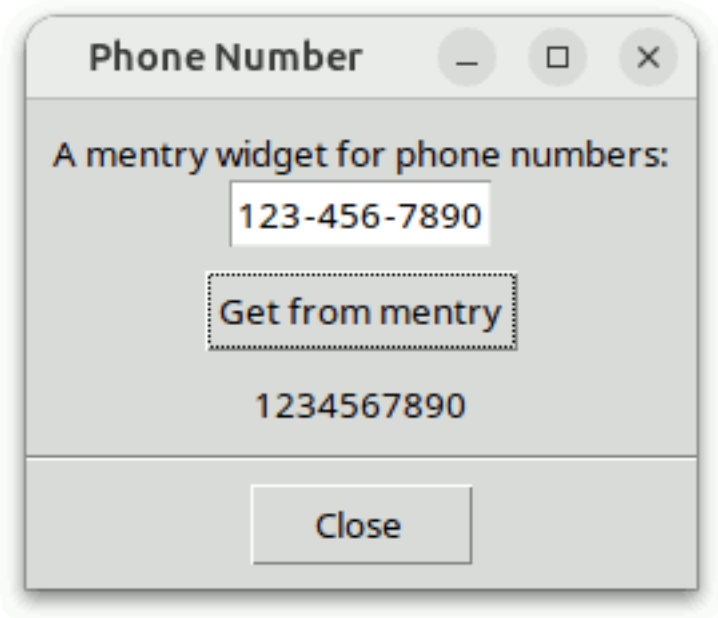
```
}

#
# Label .base.num displaying the result of getPhoneNumber
#
ttk::label .base.num -textvariable num

. . .

putPhoneNumber 1234567890 .base.f.me
focus [.base.f.me entrypath 0]
```

That's all! The resulting window has a nice theme-specific appearance:



Phone Number

The only Mentry-specific changes in the code above consist of the use of the `_tile` suffix in `mentry_tile` and a second invocation of `winfo parent`in the procedure `pastePhoneNumber` (for technical reasons, the entry components of a tile-based mentry widget are *grand*children of the mentry).

mentryDateTime

Keywords: mentryDateTime

Multi-Entry Widgets for Date and Time

For Mentry Version 4.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- The mentry::dateMentry Command
- The mentry::timeMentry Command
- The mentry::dateTimeMentry Command
- The mentry::putClockVal Command
- The mentry::getClockVal Command

The `mentry::dateMentry` Command

NAME

`mentry::dateMentry` – Create and manipulate mentry widgets for date

SYNOPSIS

```
mentry::dateMentry pathName format separator ?options?
```

DESCRIPTION

This command creates a new mentry widget `pathName` for displaying and editing a date according to the `format` argument, which must be a string of length 3, consisting of the letters `"d"` for the day (`01 - 31`), `"m"` for the month (`01 - 12`), and `"y"` or `"Y"` for the year without century (`00 - 99`) or with century (e.g., `2008`), in an arbitrary order. These field descriptor characters have the same meanings as in the `clock format`command. The `separator` argument specifies the text to be displayed in the labels separating the three entry components of the mentry widget (the most common values are the `"/"`, `"-"`, and `"."` characters). The supported `options` are the same as in the case of the `mentry::mentry` command.

The command sets the `type` attribute of the widget to the value `"Date"`, saves the value of `format` in its `format` attribute, and returns the name of the newly created widget.

DEFAULT BINDINGS

The `mentry::dateMentry` command defines four new keyboard bindings for the entry components of the mentry widget it creates: The `Up` key increments the entry's value by 1 if the latter is less than the allowed maximum for that component. Similarly, the `Down` key decrements the entry's value by 1 if the latter is greater than the allowed minimum for that component. The `Prior` key increments the entry's value by at most 10 if the latter is less than the allowed maximum for that component. Similarly, the `Next` key decrements the entry's value by at most 10 if the latter is greater than the allowed minimum for that component. If the entry is empty then all of these keys insert the component-specific minimum value into the entry.

The actions performed by the `Up` and `Down` keys can also be triggered with the aid of the mouse wheel or a two-finger gesture on the touchpad. In addition, on Mac OS X/11+ Aqua, the actions performed by the `Prior` and `Next` keys are also triggered by the `<Option-MouseWheel>` event. The same holds true on all windowing systems if the Tk version is 8.7a4 or later, where the `Option` modifier is bound to the `Option` key on Mac OS X/11+ and to the `Alt` key on Windows and X11.

KEYWORDS

mentry, widget, date

The `mentry::timeMentry` Command

NAME

`mentry::timeMentry` – Create and manipulate mentry widgets for time

SYNOPSIS

```
mentry::timeMentry pathName format separator ?options?
```

DESCRIPTION

This command creates a new mentry widget `pathName` for displaying and editing a time according to the `format` argument, which must be a string of length 2 or 3, consisting of the following field descriptor characters of the `clock format`command: `"H"` or `"I"`, followed by `"M"`, and optionally the letter `"S"`. An `"H"` as first character specifies the time format `"%H:%M"` or `"%H:%M:%S"` (i.e., with the hour between `0` and `23`), while the letter `"I"` stands for `"%I:%M %p"`or `"%I:%M:%S %p"`(i.e., with AM/PM indicator). The `separator` argument specifies the text to be displayed in the labels separating the entry components of the mentry widget (this is usually the `":"` character). The `options` are the same as in the case of the `mentry::mentry` command.

The command sets the `type` attribute of the widget to the value `"Time"`, saves the value of `format` in its `format` attribute, and returns the name of the newly created widget.

DEFAULT BINDINGS

The `mentry::timeMentry` command defines four new keyboard bindings for the entry components of the mentry widget it creates: The `Up` key increments the entry's value by 1 if the latter is less than the allowed maximum for that component. Similarly, the `Down` key decrements the entry's value by 1 if the latter is greater than the allowed minimum for that component. The `Prior` key increments the entry's value by at most 10 if the latter is less than the allowed maximum for that component. Similarly, the `Next` key decrements the entry's value by at most 10 if the latter is greater than the allowed minimum for that component. If the entry is empty then all of these keys insert the component-specific minimum value into the entry.

The actions performed by the `Up` and `Down` keys can also be triggered with the aid of the mouse wheel or a two-finger gesture on the touchpad. In addition, on Mac OS X/11+ Aqua, the actions performed by the `Prior` and `Next` keys are also triggered by the `<Option-MouseWheel>` event. The same holds true on all windowing systems if the Tk version is 8.7a4 or later, where the `Option` modifier is bound to the `Option` key on Mac OS X/11+ and to the `Alt` key on Windows and X11.

KEYWORDS

mentry, widget, time

The `mentry::dateTimeMentry` Command

NAME

`mentry::dateTimeMentry` – Create and manipulate mentry widgets for date & time

SYNOPSIS

```
mentry::dateTimeMentry pathName format dateSeparator timeSeparator
    ?options?
```

DESCRIPTION

This command creates a new mentry widget `pathName` for displaying and editing a date & time according to the `format` argument, which must be a string of length 5 or 6, with the first 3 characters consisting of the letters `"d"` for the day (`01 - 31`), `"m"` for the month (`01 - 12`), and `"y"` or `"Y"` for the year without century (`00 - 99`) or with century (e.g., `2008`), in an arbitrary order, followed by 2 or 3 further field descriptor characters of the`clock format` command, which must be: `"H"` or `"I"`, then `"M"`, and optionally the letter `"S"`. An `"H"` specifies the time format `"%H:%M"` or `"%H:%M:%S"` (i.e., with the hour between `0` and `23`), while the letter `"I"` stands for `"%I:%M %p"`or `"%I:%M:%S %p"`(i.e., with AM/PM indicator). The `dateSeparator` argument specifies the text to be displayed in the labels separating the entry components in the date part of the mentry widget (the most common values are the `"/"`, `"-"`, and `"."` characters). Similarly, the `timeSeparator` argument stands for the text to be displayed in the labels separating the entry components in the time part of mentry widget (this is usually the `":"` character). The date and time parts in turn are separated from each other by a space character. The supported `options` are the same as in the case of the `mentry::mentry` command.

The command sets the `type` attribute of the widget to the value `"DateTime"`, saves the value of `format` in its `format` attribute, and returns the name of the newly created widget.

DEFAULT BINDINGS

The `mentry::dateTimeMentry` command defines four new keyboard bindings for the entry components of the mentry widget it creates: The `Up` key increments the entry's value by 1 if the latter is less than the allowed maximum for that component. Similarly, the `Down` key decrements the entry's value by 1 if the latter is greater than the allowed minimum for that component. The `Prior` key increments the entry's value by at most 10 if the latter is less than the allowed maximum for that component. Similarly, the `Next` key decrements the entry's value by at most 10 if the latter is greater than the allowed minimum for that component. If the entry is empty then all of these keys insert the component-specific minimum value into the entry.

The actions performed by the `Up` and `Down` keys can also be triggered with the aid of the mouse wheel or a two-finger gesture on the touchpad. In addition, on Mac OS X/11+ Aqua, the actions performed by the `Prior` and `Next` keys are also triggered by the `<Option-MouseWheel>` event. The same holds true on all windowing systems if the Tk version is 8.7a4 or later, where the `Option` modifier is bound to the `Option` key on Mac OS X/11+ and to the `Alt` key on Windows and X11.

KEYWORDS

mentry, widget, date, time

The `mentry::putClockVal` Command

NAME

`mentry::putClockVal` – Output a clock value to a date, time, or date & time mentry

SYNOPSIS

```
mentry::putClockVal clockValue pathName ?-gmt boolean?
```

DESCRIPTION

This command outputs the date, time, or date & time corresponding to the integer `clockValue` to the mentry widget `pathName`, which must have been created with the `mentry::dateMentry`, `mentry::timeMentry`, or `mentry::dateTimeMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"Date"`, `"Time"`, or `"DateTime"`).

Like in the case of the `clock format`command, the optional argument pair `-gmt boolean` specifies whether the clock value is to be formatted as Greenwich Mean Time or according to the local timezone as defined by the operating environment.

KEYWORDS

mentry, widget, date, time, clock

The `mentry::getClockVal` Command

NAME

`mentry::getClockVal` – Get the clock value from a date, time, or date & time mentry

SYNOPSIS

```
mentry::getClockVal pathName ?-base clockValue? ?-gmt boolean?
```

DESCRIPTION

This command returns the clock value corresponding to the date, time, or date & time contained in the mentry widget `pathName`, which must have been created with the `mentry::dateMentry`, `mentry::timeMentry`, or `mentry::dateTimeMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"Date"`, `"Time"`, or `"DateTime"`).

Like in the case of the `clock scan`command, the optional argument pair `-base clockValue` specifies that the date contained in `clockValue` is to be used when getting the clock value from the given mentry widget, and the optional argument pair `-gmt boolean` specifies whether the clock value is to be calculated relative to Greenwich Mean Time or according to the local timezone as defined by the operating environment.

If an error occurs during the conversion, the command sets the focus to the first erronous entry component, generates an error, and returns one of the values contained in the following table:

Return Value	Meaning
"EMPTY"	Any entry component (except the one containing the seconds) is empty.
"BAD"	The value of the day, month, or hour (the latter only if the AM/PM indicator is present) is 0.
"BAD_DATE"	The <year, month, day> triple is invalid (note that the command is aware of leap years).
"BAD_YEAR"	The above triple is valid, but the conversion (made with the aid of the clock scan command) failed because of an unsupported year value (e.g., between 38 and 70).

KEYWORDS

mentry, widget, date, time, clock

mentryFixedPoint

Keywords: mentryFixedPoint

Multi-Entry Widgets for Real Numbers

in Fixed-Point Format

For Mentry Version 4.5

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- The mentry::fixedPointMentry Command
- The mentry::putReal Command
- The mentry::getReal Command

The `mentry::fixedPointMentry` Command

NAME

`mentry::fixedPointMentry` – Create and manipulate mentry widgets for real numbers in fixed-point format

SYNOPSIS

```
mentry::fixedPointMentry pathName count1 count2 ?-comma? ?options?
```

DESCRIPTION

This command creates a new mentry widget `pathName` for displaying and editing real numbers in fixed-point format, with `count1` characters before and `count2` digits after the decimal point. Both `count1` and `count2` must be positive integers. The mentry will have

two entry components, separated by a label displaying a `"."` character by default; the optional `-comma` switch changes the label text to `","`. The supported `options` are the same as in the case of the `mentry::mentry` command.

The command sets the `type` attribute of the widget to the value `"FixedPoint"` and returns the name of the newly created widget.

KEYWORDS

mentry, widget, real number

The `mentry::putReal` Command

NAME

`mentry::putReal` – Output a real number to a mentry of type `"FixedPoint"`

SYNOPSIS

```
mentry::putReal number pathName
```

DESCRIPTION

This command outputs the real number `number` to the mentry widget `pathName`, which must have been created with the `mentry::fixedPointMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"FixedPoint"`).

The command generates an error if the number does not fit into the widget because it has too many characters before the decimal point.

KEYWORDS

mentry, widget, real number

The `mentry::getReal` Command

NAME

`mentry::getReal` – Get a real number from a mentry of type `"FixedPoint"`

SYNOPSIS

```
mentry::getReal pathName
```

DESCRIPTION

This command returns the number contained in the mentry widget `pathName`, which must have been created with the `mentry::fixedPointMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"FixedPoint"`).

If both entry components of the widget are empty, the command sets the focus to the first entry component, generates an error, and returns the string `"EMPTY"`.

KEYWORDS

mentry, widget, real number

mentryIPAddr

Keywords: mentryIPAddr

Multi-Entry Widgets for IP Addresses

For Mentry Version 4.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- The mentry::ipAddrMentry Command
- The mentry::putIPAddr Command
- The mentry::getIPAddr Command

The `mentry::ipAddrMentry` Command

NAME

`mentry::ipAddrMentry` – Create and manipulate mentry widgets for IP addresses

SYNOPSIS

```
mentry::ipAddrMentry pathName ?options?
```

DESCRIPTION

This command creates a new mentry widget `pathName` for displaying and editing IP addresses in the standard dotted-decimal notation. The supported `options` are the same as in the case of the `mentry::mentry` command.

The command sets the `type` attribute of the widget to the value `"IPAddr"` and returns the name of the newly created widget.

DEFAULT BINDINGS

The `mentry::ipAddrMentry` command defines five new keyboard bindings for the entry components of the mentry widget it creates: The `Up` key increments the entry's value by 1 if the latter is less than 255. Similarly, the `Down` key decrements the entry's value by 1 if the latter is greater than 0. The `Prior` key increments the entry's value by at most 10 if the latter is less than 255. Similarly, the `Next` key decrements the entry's value by at most 10 if the latter is greater than 0. If the entry is empty then all of these keys insert the value `0` into the entry. Finally, `Control-v` (`Command-v` on the Mac) pastes the current contents of the clipboad into the mentry, provided that it is a valid IP address.

The actions performed by the `Up` and `Down` keys can also be triggered with the aid of the mouse wheel or a two-finger gesture on the touchpad. In addition, on Mac OS X/11+ Aqua, the actions performed by the `Prior` and `Next` keys are also triggered by the `<Option-MouseWheel>` event. The same holds true on all windowing systems if the Tk version is 8.7a4 or later, where the `Option` modifier is bound to the `Option` key on Mac OS X/11+ and to the `Alt` key on Windows and X11.

KEYWORDS

mentry, widget, IP address

The `mentry::putIPAddr` Command

NAME

`mentry::putIPAddr` – Output an IP address to a mentry of type `"IPAddr"`

SYNOPSIS

```
mentry::putIPAddr address pathName
```

DESCRIPTION

This command outputs the IP address `address` to the mentry widget `pathName`, which must have been created with the `mentry::ipAddrMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"IPAddr"`).

The command generates an error if the address is invalid.

KEYWORDS

mentry, widget, IP address

The `mentry::getIPAddr` Command

NAME

`mentry::getIPAddr` – Get an IP address from a mentry of type `"IPAddr"`

SYNOPSIS

```
mentry::getIPAddr pathName
```

DESCRIPTION

This command returns the IP address contained in the mentry widget `pathName`, which must have been created with the `mentry::ipAddrMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"IPAddr"`).

If any entry component of the widget is empty, the command sets the focus to the first such entry, generates an error, and returns the string `"EMPTY"`.

KEYWORDS

mentry, widget, IP address

mentryIPv6Addr

Keywords: mentryIPv6Addr

Multi-Entry Widgets for IPv6 Addresses

For Mentry Version 4.5

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- The mentry::ipv6AddrMentry Command
- The mentry::putIPv6Addr Command
- The mentry::getIPv6Addr Command

The `mentry::ipv6AddrMentry` Command

NAME

`mentry::ipv6AddrMentry` – Create and manipulate mentry widgets for IPv6 addresses

SYNOPSIS

```
mentry::ipv6AddrMentry pathName ?options?
```

DESCRIPTION

This command creates a new mentry widget `pathName` for displaying and editing IPv6 addresses in the standard hexadecimal notation (where the eight groups of one to four hexadecimal digits are separated from each other by o colon). The supported `options` are the same as in the case of the `mentry::mentry` command.

The command sets the `type` attribute of the widget to the value `"IPv6Addr"` and returns the name of the newly created widget.

DEFAULT BINDINGS

The `mentry::ipv6AddrMentry` command defines five new keyboard bindings for the entry components of the mentry widget it creates: The `Up` key increments the entry's value by 1 if the latter is less than 65535. Similarly, the `Down` key decrements the entry's value by 1 if the latter is greater than 0. The `Prior` key increments the entry's value by at most 10 if the latter is less than 65535. Similarly, the `Next` key decrements the entry's value by at most 10 if the latter is greater than 0. If the entry is empty then all of these keys insert the value `0` into the entry. Finally, `Control-v` (`Command-v` on the Mac) pastes the current contents of the clipboad into the mentry, provided that it is a valid IPv6 address.

The actions performed by the `Up` and `Down` keys can also be triggered with the aid of the mouse wheel or a two-finger gesture on the touchpad. In addition, on Mac OS X/11+ Aqua, the actions performed by the `Prior` and `Next` keys are also triggered by the `<Option-MouseWheel>` event. The same holds true on all windowing systems if the Tk version is 8.7a4 or later, where the `Option` modifier is bound to the `Option` key on Mac OS X/11+ and to the `Alt` key on Windows and X11.

KEYWORDS

mentry, widget, IPv6 address

The `mentry::putIPv6Addr` Command

NAME

`mentry::putIPv6Addr` – Output an IPv6 address to a mentry of type `"IPv6Addr"`

SYNOPSIS

```
mentry::putIPv6Addr address pathName
```

DESCRIPTION

This command outputs the IPv6 address `address` to the mentry widget `pathName`, which must have been created with the `mentry::ipv6AddrMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"IPv6Addr"`).

The address is expected to consist of up to eight colon-separated groups of one to four hexadecimal digits each. in case of less than eight groups, the address must include exactly one double-colon (`::`) (representing consecutive groups of zeros). The command generates an error if the address is invalid.

KEYWORDS

mentry, widget, IPv6 address

The `mentry::getIPv6Addr` Command

NAME

`mentry::getIPv6Addr` – Get an IPv6 address from a mentry of type `"IPv6Addr"`

SYNOPSIS

```
mentry::getIPv6Addr pathName
```

DESCRIPTION

This command returns the IPv6 address contained in the mentry widget `pathName`, which must have been created with the `mentry::ipv6AddrMentry` command (this is checked by examining the widget's `type` attribute, which must have the value `"IPv6Addr"`).

If any entry component of the widget is empty, the command sets the focus to the first such entry, generates an error, and returns the string `"EMPTY"`.

KEYWORDS

mentry, widget, IPv6 address

mentryThemes

Keywords: mentryThemes

Commands Related to Tile Themes

For Mentry Version 4.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- Overview
- The mentry::setTheme Command

- The mentry::getCurrentTheme Command
- The mentry::getThemes Command
- The mentry::setThemeDefaults Command

Overview

The commands described in this reference page should only be invoked when using the package Mentry_tile. They enable you to set and query the current theme, to retrieve a list of the available themes, and to make sure that your widgets will have a theme-specific appearance.

The `mentry::setTheme` Command

NAME

`mentry::setTheme` – Set the current theme

SYNOPSIS

```
mentry::setTheme theme
```

DESCRIPTION

This command sets the current theme to `theme`, loading it if necessary. It is simply an alias for `ttk::setTheme` or `tile::setTheme`, depending on the tile version loaded into the interpreter. (The `tile::setTheme` command was renamed to `ttk::setTheme` in tile version 0.8.)

Being just an alias for a tile library procedure, the `mentry::setTheme` command does exactly the same as the original one: It loads the package implementing the given theme if needed, sets the theme to the specified one, and saves the latter in the variable `ttk::currentTheme` or `tile::currentTheme`, depending on the current tile version.

KEYWORDS

mentry, theme, tile

The `mentry::getCurrentTheme` Command

NAME

`mentry::getCurrentTheme` – Get the current theme

SYNOPSIS

```
mentry::getCurrentTheme
```

DESCRIPTION

This command returns the name of the current theme by invoking the `ttk::style theme use`command. If an old tile version is being used which doesn't yet support this method then the command returns the value of the variable `ttk::currentTheme` or `tile::currentTheme`, depending on the tile version. (The namespace containing the variable `currentTheme` was changed in tile version 0.8 from `tile` to `ttk`.)

KEYWORDS

mentry, theme, tile

The `mentry::getThemes` Command

NAME

`mentry::getThemes` – Get the themes registered in the package database

SYNOPSIS

```
mentry::getThemes
```

DESCRIPTION

This command returns a list of the themes registered in the package database. It is simply an alias for `ttk::themes` or `tile::availableThemes`, depending on the tile version loaded into the interpreter. (The `tile::availableThemes` command was renamed to `ttk::themes` in tile version 0.8.)

KEYWORDS

mentry, theme, tile

The `mentry::setThemeDefaults` Command

NAME

`mentry::setThemeDefaults` – Set theme-specific default values of some mentry configuration options

SYNOPSIS

```
mentry::setThemeDefaults
```

DESCRIPTION

This command populates the array `mentry::themeDefaults` with theme-specific default values of a series of Mentry configuration options. The array names are the command-line names of the options, and the corresponding array values are the default values of these configuration options for the currently set tile theme.

The options whose names and values are written into the array `mentry::themeDefaults` are: `-background`, `-foreground`, and `-font`. In addition, the command sets some other array elements to theme-specific default values, needed for internal purposes, like updating the background and foreground colors of the label components of a mentry widget in `disabled` or `readonly` state; the corresponding array names are: `-disabledbackground`, `-disabledforeground`, and `-readonlybackground`. Finally, the array values associated with the names `-selectbackground`, `-selectforeground`, and `-selectborderwidth` are not used by Mentry_tile, but might prove useful for other purposes, as described below.

The `mentry::setThemeDefaults` command is invoked by Mentry_tile automatically at application start time and when the `<<ThemeChanged>>` virtual event is sent to the main window. It is also invoked on macOS 10.14 (Mojave) and later whenever the main window receives a `<<LightAppearance>>` (formerly: `<<LightAqua>>`) or `<<DarkAppearance>>` (formerly: `<<DarkAqua>>`) virtual event, if the current theme is `aqua`. In these cases, after invoking the `mentry::setThemeDefaults` command,

Mentry_tile sends the virtual event <<MentryThemeDefaultsChanged>> to the main window and reconfigures all mentry widgets of the application, using the new default values of those options that were not set explicitly to values different from the corresponding defaults.

On macOS 10.14 (Mojave) and later, the virtual events <<LightAppearance>> and <<DarkAppearance>> (or <<LightAqua>> and <<DarkAqua>>) are sent by Tk 8.6.10 and later to all toplevel widgets whenever the system appearance toggles between Light Mode and Dark Mode or the Accent Color in the General Preferences changes. Tk versions 8.6.11 and later send these events on macOS 10.15 (Catalina) and later also when changing the Highlight Color in the General Preferences.

Besides being used by Mentry_tile, this command can also be invoked in application code, although with current Mentry_tile versions there is no need for calling it explicitly.

By using the default values contained in the array mentry::themeDefaults, you can make sure that classical Tk widgets, e.g., text, will have a theme-specific appearance, just like the tile widgets. For example, you can add some common configuration options to the option database as follows:

```
if {[tk windowingsystem] eq "x11"} {
    option add *Text.background   $mentry::themeDefaults(-background)
    option add *Text.foreground   $mentry::themeDefaults(-foreground)
    option add *selectBackground
    $mentry::themeDefaults(-selectbackground)
    option add *selectForeground
    $mentry::themeDefaults(-selectforeground)
}
option add *selectBorderWidth
    $mentry::themeDefaults(-selectborderwidth)
option add *Font                  $mentry::themeDefaults(-font)
```

You can also make use of the virtual event <<MentryThemeDefaultsChanged>> to update the appearance of your Tk widgets according to the new values contained in the array mentry::themeDefaults, by binding to the dedicated binding tag MentryMain associated with the main window:

```
bind MentryMain <<MentryThemeDefaultsChanged>> ...
```

The binding tag MentryMain is assigned by Mentry_tile to the main window and used internally for handling the virtual events <<ThemeChanged>>, <<LightAppearance>> (or <<LightAqua>>), and <<DarkAppearance>> (or <<DarkAqua>>). As shown above, applications can make use of it, e.g., to handle the virtual event <<MentryThemeDefaultsChanged>>.

KEYWORDS

mentry, theme, tile

mentryWidget

Keywords: mentryWidget

The `mentry::mentry` Command

For Mentry Version 4.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- Quick Reference
- Detailed Reference

Quick Reference

NAME

`mentry::mentry` – Create and manipulate multi-entry widgets

SYNOPSIS

```
mentry::mentry pathName ?options?
```

STANDARD OPTIONS

```
-borderwidth          -highlightcolor      -relief
-highlightbackground  -highlightthickness
```

OPTIONS FOR ALL ENTRY AND LABEL COMPONENTS

```
-background  -cursor  -font  -foreground
```

OPTIONS FOR ALL ENTRY COMPONENTS

```
-exportselection    -insertwidth        -selectforeground
-insertbackground   -invalidcommand     -show
-insertborderwidth  -justify            -state
-insertofftime      -selectbackground   -validate
-insertontime       -selectborderwidth  -validatecommand
```

WIDGET-SPECIFIC OPTIONS

```
-body {width ?text width text ...?}

-disabledbackground color

-disabledforeground color

-readonlybackground color
```

```
  -takefocus 0|1|""|command

  -textvariable array
```

INDICES

```
number  end
```

WIDGET COMMAND

```
  pathName adjustentry index string1 ?string2?

  pathName attrib ?name? ?value name value ...?

  pathName cget option

  pathName clear first ?last?

  pathName configure ?option? ?value option value ...?

  pathName entries

  pathName entrycount

  pathName entrylimit index

  pathName entrypath index

  pathName getarray array

  pathName getlist

  pathName getstring

  pathName hasattrib name

  pathName isempty ?index?

  pathName isfull ?index?

  pathName labelcount

  pathName labelpath index

  pathName labels

  pathName put startIndex ?string string ...?
```

```
pathName setentryextrawidth index amount

pathName setentryfont index font

pathName setentrywidth index width

pathName unsetattrib name
```

KEYWORDS

mentry, widget, entry, label

Detailed Reference

NAME

`mentry::mentry` – Create and manipulate multi-entry widgets

SYNOPSIS

```
mentry::mentry pathName ?options?
```

DESCRIPTION

The `mentry::mentry` command creates a new window named `pathName` and of the class `Mentry`, and makes it into a **mentry** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the mentry such as its colors, font, and components. The `mentry::mentry` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s parent must exist.

The word *mentry* is an abbreviation of *multi-entry*. A mentry is a mega-widget consisting of any number of entries separated by labels, all embedded in a frame. Due to appropriately chosen configuration options for the entries and labels, a mentry widget looks like one single entry containing preinserted text pieces that cannot be moved, changed, or deleted by the user.

In the Mentry package both the entry and label components are direct children of the mentry widget. In the Mentry_tile package only the labels are direct children, while the entry components are grandchildren of the mentry (for technical reasons, the entry components of a tile-based mentry widget are embedded in frame widgets, which in turn are children of the mentry).

The initial width of each entry component (specified by the `-body` configuration option) also determines the maximal number of characters that can be inserted into it. This is achieved with the aid of the `wcb::callback` command, by registering the `wcb::checkEntryLen` before-`insert` callback with the entry component. In order to keep this callback, additional before-`insert` callbacks (if needed) should only be registered with the `wcb::cbappend` and `wcb::cbprepend` commands. When reaching this limit in an entry having the input focus, the latter is set automatically to the next enabled entry component, the contents of that widget is selected, and the insertion cursor is set to its end. This is made sure by defining an appropriate after-`insert` callback, which must be kept to be the last such callback for the

entry. For this reason, additional after-`insert` callbacks (if needed) should only be registered with the `wcb::cbprepend` command. The same command should be used exclusively to define after-`motion` callbacks (if needed), because `mentry::mentry` registers one that must be the last after-`motion` callback for each entry component.

Each mentry widget may have any number of **attributes**, which can be used in commands that create or manipulate mentry widgets for particular purposes. For example, each date mentry created by the `mentry::dateMentry` command has a `type` attribute having the value `"Date"` and a `format` attribute specifying the meanings of its components. The values of these attributes are queried and used in the bodies of the procedures `mentry::putClockVal` and `mentry::getClockVal`.

STANDARD OPTIONS

```
-borderwidth          -highlightcolor      -relief
-highlightbackground  -highlightthickness
```

See the **options** manual entry for details on the standard Tk widget options. These options are only supported by the Mentry package, but not by Mentry_tile.

OPTIONS FOR ALL ENTRY AND LABEL COMPONENTS

```
-background  -cursor  -font  -foreground
```

These options (described in the **options** manual entry) specify the values of the options of the same names for all entry and label components of the given multi-entry widget. These values may be changed for an individual component by using the `configure` subcommand of the Tcl command corresponding to that component. However, applying this method to change the font of an entry of a tile-based mentry widget might break the latter's appearance (this has tile-related technical reasons). To avoid this problem, it is recommended to use the `setentryfont` subcommand of the Tcl command associated with the mentry widget for changing the value of the `-font` option of one of its entry components.

When using the package Mentry_tile, these options have theme-specific default values. The common abbreviations `-bg` (for `-background`) and `-fg` (for `-foreground`) are supported for both classical and tile-based mentry widgets.

REMARK: Please take into account that for some themes (like `Arc`, `plastik`, `tileqt`, `vista`, and `xpnative`), the `-background` option will be ignored for the entry components of a tile-based mentry widget. The same holds true for the `aqua` theme in dark mode and with some Tk versions in the light appearance, too.

OPTIONS FOR ALL ENTRY COMPONENTS

```
-exportselection    -insertwidth        -selectforeground
-insertbackground   -invalidcommand     -show
-insertborderwidth  -justify            -state
-insertofftime      -selectbackground   -validate
-insertontime       -selectborderwidth  -validatecommand
```

These options specify the values of the options of the same names for all entry components

of the given multi-entry widget. These values may be changed for an individual entry component by using the `configure` subcommand of the Tcl command corresponding to that entry. The options `-insertbackground`, `-insertborderwidth`, `-insertofftime`, `-insertontime`, `-insertwidth`, `-selectbackground`, `-selectborderwidth`, and `-selectforeground` are only supported by the Mentry package, but not by Mentry_tile. The common abbreviations `-invcmd` (for `-invalidcommand`) and `-vcmd` (for `-validatecommand`) are supported for both classical and tile-based mentry widgets.

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-body
Database Name:	body
Database Class:	Body

Specifies the initial widths of the entry components of the given multi-entry widget, as well as the texts to be displayed in the labels separating the entries. The value of this option is viewed as a list. Each odd-numbered element is interpreted as the value of the `-width` configuration option of an entry component, while each even-numbered element represents the value of the `-text` configuration option of a label child. That is, each entry width must be followed by a label text, except for the last one, for which a corresponding label text is optional. The order of the list elements determines the creation order of the components; these will be packed in a row, without any horizontal padding. The initial width of each entry also determines the number of characters that can be inserted into it; this maximal length remains unchanged even if the entry's width is set to a new value later on.

The width of an entry component and the text displayed in a label may be changed by using the `configure` subcommand of the Tcl command corresponding to that component. However, applying this method to change the width of of an entry of a tile-based mentry widget might break the latter's appearance (this has tile-related technical reasons). To avoid this problem, it is recommended to use the `setentrywidth` subcommand of the Tcl command associated with the mentry widget for changing the value of the `-width` option of one of its entry components.

Command-Line Name:	-disabledbackground
Database Name:	disabledBackground
Database Class:	DisabledBackground

This option specifies the background color to use for all entry and label components of the given multi-entry widget when it is disabled. If the value of this option is an empty string then the normal background color is used. This option is only supported by the Mentry package, but not by Mentry_tile.

Command-Line Name:	-disabledforeground

Database Name:	disabledForeground
Database Class:	DisabledForeground

This option specifies the foreground color to use for all entry and label components of the given multi-entry widget when it is disabled. If the value of this option is an empty string then the normal foreground color is used. This option is only supported by the Mentry package, but not by Mentry_tile.

Command-Line Name:	-readonlybackground
Database Name:	readonlyBackground
Database Class:	ReadonlyBackground

This option specifies the background color to use for all entry and label components of the given multi-entry widget when it is readonly. If the value of this option is an empty string then the normal background color is used. This option is only supported by the Mentry package, but not by Mentry_tile.

Command-Line Name:	-takefocus
Database Name:	takeFocus
Database Class:	TakeFocus

This option determines whether the widget accepts the focus during keyboard traversal. It is almost identical to the standard option of the same name (see the **options** manual entry for details). The only difference is that not the widget itself but its first enabled entry component will receive the focus during keyboard traversal with the standard keys (`Tab` and `Shift-Tab`). See the DEFAULT BINDINGS section below for information on how to move the focus from one entry component to another one using the keyboard.

Command-Line Name:	-textvariable
Database Name:	textvariable
Database Class:	Variable

If the value of this option is not an empty string then it must be an array. In this case, for each entry component of the given multi-entry widget the value of the array element with the index of the entry as element name specifies the value of the `-textvariable` option for that entry. The entry indices start with `0`. For example, specifying `-textvariable arr`for a multi-entry widget having two entry components is equivalent to specifying `-textvariable arr(0)`for the first and `-textvariable arr(1)`for the second entry of that widget.

INDICES

Some of the widget commands for mentries take an index as argument. An index specifies a particular entry or label component of the mentry, in any of the following ways:

number	Specifies the component as a numerical index, where 0 corresponds to the first entry or label component.
end	Indicates the last entry or label component of the mentry.

The word `end` can be abbreviated as `en` or `e`. Out-of-range indices are automatically rounded to the nearest legal value.

WIDGET COMMAND

The `mentry::mentry` command creates a new Tcl command whose name is `pathName`. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

`option` and the `arg`s determine the exact behavior of the command. The following commands are possible for mentry widgets:

`pathName adjustentry index string1 ?string2?`

Increases the width in pixels of the entry component identified by `index` if needed, to make it just large enough for texts of the entry-specific maximal length, consisting of characters contained in the `string1` argument, excepting at most one character, which may be contained in the optional `string2` argument. Returns an empty string.

For example, when constructing a mentry widget whose components will accept up to two uppercase hexadecimal digits only, by passing the string `"0123456789ABCDEF"` to this subcommand you can make sure that the entry components will have optimal widths, just large enough to display not only the decimal digits, but also up to two letters in the range `A` - `F` (which need more room than the decimal digits when using a proportionally-spaced font). It is recommended to invoke this subcommand even if the allowed characters are restricted to the digits `0` - `9` only, because in some fonts not all decimal digits have the same width.

An example for the case where both string arguments are used is a mentry widget for (signed) real numbers in fixed-point format. For its first entry component (displaying the integer part of a real number), this subcommand should be invoked with `"0123456789"` as its `string1` argument and `"+-"` as its `string2` argument, being that the entry's first character can be not only a decimal digit but also the sign `"+"` or `"-"`. (Note that in most proportional fonts, the width of the `"+"` character is larger than that of `"0"`.)

`pathName attrib ?name? ?value name value ...?`

Queries or modifies the attributes of the widget. If no `name` is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for `pathName`. If `name` is specified with no `value`, then the command returns the value of the one named attribute, or an empty string if no corresponding value exists (you can use the `hasattrib` subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more `name-value` pairs are specified, then the

command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. `name` may be an arbitrary string.

`pathName cget option`
Returns the current value of the configuration option given by `option`, which may have any of the values accepted by the `mentry::mentry` command.

`pathName clear first ?last?`
Erases the contents of one or more entry components of the widget. `first` and `last` are indices specifying the first and last entries in the range to clear. If `last` is not specified then it defaults to `first`, i.e., a single entry is cleared. The return value is an empty string.

`pathName configure ?option? ?value option value ...?`
Queries or modifies the configuration options of the widget. If no `option` is specified, the command returns a list describing all of the available options for `pathName` (see `Tk_ConfigureInfo` for information on the format of this list). If `option` is specified with no `value`, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no `option` is specified). If one or more `option-value` pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. `option` may have any of the values accepted by the `mentry::mentry` command.

`pathName entries`
Returns a list containing the path names of all entry components of the widget.

`pathName entrycount`
Returns the number of all entry components of the widget.

`pathName entrylimit index`
Returns the number of characters that can be inserted into the entry component identfied by `index`.

`pathName entrypath index`
Returns the path name of the entry component identfied by `index`.

`pathName getarray array`
For each entry component of the widget, the command assigns the string contained in the entry to the element of `array` having the index of the entry as element name. The entry indices start with `0`. The return value is an empty string.

`pathName getlist`
Returns a list containing the strings of all entry components of the widget.

`pathName getstring`
Returns a string built by concatenating the strings contained in the entry components of the widget with the texts displayed in the labels separating the entries, in the creation order of the components, as given by the `-body` configuration option. That is, the string of the first entry is followed by the text of the first label, which in turn is followed by the string of the second entry, and so on.

`pathName hasattrib name`
Returns `1` if the attribute `name` exists and `0` otherwise.

`pathName isempty ?index?`
If `index` is specified then the command returns the value `1` if the string contained in the entry component identfied by `index` is empty and `0` otherwise. If this optional argument is not present then the command returns the value `1` if the strings contained in the entry

components of the widget are all empty and `0` otherwise.

`pathName isfull ?index?`
If `index` is specified then the command returns the value `1` if the length of the string contained in the entry component identfied by `index` equals the number of characters that can be inserted into the entry and `0` otherwise. If this optional argument is not present then the command returns the value `1` if the lengths of the strings contained in the entry components of the widget all equal the numbers of characters that can be inserted into the respective entries (i.e., if all entry components are full) and `0` otherwise.

`pathName labelcount`
Returns the number of all label children of the widget.

`pathName labelpath index`
Returns the path name of the label child identfied by `index`.

`pathName labels`
Returns a list containing the path names of all label children of the widget.

`pathName put startIndex ?string string ...?`
Replaces the texts of the entry components whose indices are greater than or equal to `startIndex` with the corresponding `string` arguments by using the `delete` and `insert` operations, until either the entries or the `string`s are consumed. If one of these subcommands gets canceled by some before-callback in any entry component then the command restores the original contents of all affected entries and returns the value `0`, otherwise it returns `1`. In both cases, the command keeps the position of the insertion cursor in all entry components.

`pathName setentryextrawidth index amount`
Specifies additonal width to provide for the entry component identified by `index`. `amount` may have any of the standard forms for screen distances (e.g., a number of pixels). The return value is an empty string.

Notice that changing the width of an entry component has no impact on the number of characters that can be inserted into it, which remains set to the corresponding component of the value of the mentry's `-body` option.

`pathName setentryfont index font`
Sets the font of the entry component identified by `index` to the value given by `font`. This is done by invoking the `configure` subcommand of the Tcl command corresponding to the entry. When using the package Mentry_tile, this command also performs some additional steps necessary for updating the appearance of the tile-based mentry widget. The return value is an empty string.

`pathName setentrywidth index width`
Sets the width of the entry component identified by `index` to the number of characters given by `width`. This is done by invoking the `configure` subcommand of the Tcl command corresponding to the entry. When using the package Mentry_tile, this command also performs some additional steps necessary for updating the appearance of the tile-based mentry widget. The return value is an empty string.

Notice that changing the width of an entry component has no impact on the number of characters that can be inserted into it, which remains set to the corresponding component of

the value of the mentry's `-body` option.

`pathName unsetattrib name`
Unsets the attribute `name`. Returns an empty string.

DEFAULT BINDINGS

The `mentry::mentry` command partially redefines the bindings of the components of the mentry widget it creates:

1. Clicking mouse button 1 in a label child positions the insertion cursor to the end of the last enabled entry preceding the label, sets the input focus to that entry, and clears any selection in it.
2. If in a non-empty entry component a character is typed that is contained in the label following that entry (if any), then the focus is moved to the next enabled entry component, the contents of that entry are selected, and the insertion cursor is set to its end.
3. At the beginning of an entry component that is not the first enabled entry within the widget, the `Left` key clears the selection in that entry, moves the focus to the previous enabled entry component, and sets the insertion cursor to the end of that widget. Similarly, at the end of an entry component that is not the last enabled entry within the widget, the `Right` key clears the selection in that entry, moves the focus to the next enabled entry component, and sets the insertion cursor to the beginning of that widget. If `tk_strictMotif` is false then `Control-b` and `Control-f` behave the same as `Left` and `Right`, respectively.
4. In an entry component that is not the first enabled entry within the widget, `Control-Left` moves the focus to the previous enabled entry component, selects the contents of that widget, and sets the insertion cursor to its end; if there is no enabled entry component to the left of the current one then `Control-Left` moves the insertion cursor to the beginning of the current entry and clears the selection in that widget. Similarly, in an entry component that is not the last enabled entry within the widget, `Control-Right` moves the focus to the next enabled entry component, selects the contents of that widget, and sets the insertion cursor to its end; if there is no enabled entry component to the right of the current one then `Control-Right` moves the insertion cursor to the end of the current entry and clears the selection in that widget. If `tk_strictMotif` is false then `Meta-b` and `Meta-f` behave the same as `Control-Left` and `Control-Right`, respectively.
5. The `Home` key clears the selection in the current entry widget, moves the focus to the first enabled entry component, and sets the insertion cursor to the beginning of that widget. Similarly, the `End` key clears the selection in the current entry widget, moves the focus to the last enabled entry component, and sets the insertion cursor to the end of that widget. If `tk_strictMotif` is false then `Control-a` and `Control-e` behave the same as `Home` and `End`, respectively.
6. `Shift-Home` moves the focus to the first enabled entry component, sets the insertion cursor to the beginning of that widget, and either extends the selection to that position, or clears the selection in the current entry and selects the contents of the new widget, depending upon whether the current entry is the first enabled entry component or not. Similarly, `Shift-End` moves the focus to the last enabled entry component, sets the insertion cursor to the end of that widget, and either extends the selection to that position, or clears the selection in the current entry and selects the contents of the new widget, depending upon whether the current entry is the last enabled entry component or not.
7. The `BackSpace` key deletes the selection if there is one in the current entry. Otherwise, it

deletes either the character to the left of the insertion cursor in the current entry, or the last character of the previous enabled entry component. The latter takes place at the beginning of an entry component that is not the first enabled entry within the widget; in this case, the `BackSpace` key also moves the focus to the previous enabled entry component and sets the insertion cursor to its end. If `tk_strictMotif` is false then `Control-h` behaves the same as `BackSpace`.

8. If `tk_strictMotif` is false then `Meta-d` deletes all characters to the right of the insertion cursor within the current entry, while `Meta-BackSpace` and `Meta-Delete` delete either all characters to the left of the insertion cursor in the current entry, or the contents of the previous enabled entry component. The latter takes place at the beginning of an entry component that is not the first enabled entry within the widget; in this case, `Meta-BackSpace` and `Meta-Delete` also clear the selection in the current entry widget and move the focus to the previous enabled entry component.

KEYWORDS

mentry, widget, entry, label

wcbRef

Keywords: wcbRef

Wcb Command Reference

For Wcb Version 4.2

by

Csaba Nemethi

csaba.nemethi@t-online.de

Contents

- The wcb::callback Command
- The wcb::cbappend Command
- The wcb::cbprepend Command
- The wcb::cancel Command
- The wcb::canceled Command
- The wcb::extend Command
- The wcb::replace Command
- The wcb::pathname Command
- The wcb::changeEntryText Command
- The wcb::postInsertEntryLen Command
- The wcb::postInsertEntryText Command
- The wcb::postDeleteEntryText Command
- Before-insert Callbacks for entry, spinbox, and combobox Widgets
- Before-insert Callbacks for text and ctext Widgets

The `wcb::callback` Command

NAME

`wcb::callback` – Retrieve, set, and remove widget callbacks

SYNOPSIS

```
wcb::callback widgetName before|after option ?callback callback ...?
```

DESCRIPTION

Retrieves, sets, or removes the callbacks for the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, Ttk combobox, listbox, tablelist, Ttk treeview, text, or ctext widget `widgetName`, the argument `before` or `after`, and the command corresponding to `option`. The values of the `option` argument can be:

- `insert`, `delete`, or `motion`, for a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, Ttk combobox, text, or ctext widget;
- `replace`, for a text or ctext widget;
- `activate`, for a listbox, tablelist, or Ttk treeview widget;
- `selset` or `selclear`, for a listbox, tablelist, Ttk treeview, text, or ctext widget;
- `seladd` or `seltoggle`, for a Ttk treeview widget;
- `cellselset` or `cellselclear`, for a tablelist or Ttk treeview widget;
- `cellseladd` or `cellseltoggle`, for a Ttk treeview widget;
- `activatecell`, for a tablelist widget.

If no arguments after the `option` parameter are specified, then the procedure just returns the current before- or after-callback list, respectively, for the given widget operation.

Otherwise:

- If at least one of the arguments following the `option` parameter is a nonempty string, then:
- If all arguments following the `option` parameter are empty, then the procedure unregisters all the corresponding before- or after-callbacks for the given widget and returns an empty string.
  - if called for the first time for this widget with at least one nonempty argument following the `option` parameter, then the procedure renames the Tcl command `::widgetName` to `::_widgetName` and builds a new procedure `::widgetName`, in which the execution of the widget operations associated with the above values of `option` is preceded by invocations of the corresponding before-callbacks and followed by calls to the corresponding after-callbacks, in the global scope;
  - it sets the callback list to the one built from these arguments and returns the new list.

When a callback is invoked for the widget `widgetName`, the name of the **proxy command** `::_widgetName` and the command arguments are automatically appended to it as parameters. Use the `wcb::pathname` command to retrieve the original widget path name.

The following table shows the widget subcommands corresponding to the above values of `option`, together with the arguments of these subcommands:

Widget	option	Subcommand	Arguments
Tk entry, ttk::entry, BWidget Entry, Tk spinbox, ttk::spinbox, or ttk::combobox	insert	insert	index string
delete	delete	from ?to?	
motion	icursor	index	
listbox	activate	activate	index
selset	selection set	first ?last?	
selclear	selection clear	first ?last?	
tablelist::tablelist	activate	activate	index
selset	selection set	first ?last? (indexList or firstIndex lastIndex)	
selclear	selection clear	first ?last? (indexList or firstIndex lastIndex)	
activatecell	activatecell	cellIndex	
cellselset	cellselection set	first ?last? (cellIndexList or firstCell lastCell)	
cellselclear	cellselection clear	first ?last? (cellIndexList or firstCell lastCell)	
ttk::treeview	activate	focus	item
selset	selection set	itemList	
seladd	selection add	itemList	
selclear	selection remove	itemList	
seltoggle	selection toggle	itemList	
cellselset	cellselection set	first ?last? (cellList or firstCell lastCell)	
cellseladd	cellselection add	first ?last? (cellList or firstCell lastCell)	
cellselclear	cellselection remove	first ?last? (cellList or firstCell lastCell)	

Widget	option	Subcommand	Arguments
cellseltoggle	cellselection toggle	first ?last? (cellList or firstCell lastCell)	
text or ctext	insert	insert	index string ?tagList string tagList ...?
delete	delete	from ?to?	
replace	replace	from to string ?tagList string tagList ...?	
motion	mark set insert	index	
selset	tag add sel	from ?to from to ...?	
selclear	tag remove sel	from ?to from to ...?	

REMARKS:

1. You may abbreviate the words `before`, `after`, `insert`, `delete`, `replace`, `motion`, and `activate` (the latter not for tablelist widgets) to a minimum of one character. Similarly, the first four characters of the words `selset`, `selclear`, `seladd`, and `seltoggle`, the first eight characters of `cellselset`, `cellselclear`, `cellseladd`, and `cellseltoggle`, and the first nine characters of `activatecell` are sufficient for Wcb to recognize these options.
2. After a successful invocation of this command with at least one nonempty callback following the `option` argument, you can use either the new procedure `widgetName` or the proxy command `::_widgetName` to perform any valid operation on the widget `widgetName`. Use the proxy command `::_widgetName` if you want to prevent the callbacks from being invoked when executing the respective widget subcommand.
3. When destroying a widget `widgetName` for which `wcb::callback` has replaced the corresponding Tcl command with a new procedure, the proxy command `::_widgetName` is deleted automatically by the Tcl interpreter (this is not true in the case of a BWidget Entry, tablelist, or ctext widget). The new widget procedure `widgetName` would persist, but Wcb arranges for it to be deleted from within a cleanup script bound to the `<Destroy>` event. This cleanup script is associated with a binding tag called `WcbCleanup`, which is appended to the list of binding tags of the widget the first time when registering some callbacks for it. (In the case of a BWidget Entry, tablelist, or ctext widget, this script also deletes the proxy command `::_widgetName`.)
4. The cleanup script mentioned above also unregisters all callbacks defined for `widgetName`, thus ensuring that a widget with the same path created later will not inherit them from the widget just deleted (this can be important in some applications). For this reason, you should be careful not to remove `WcbCleanup` from the list of binding tags of the given widget!

KEYWORDS

callback, widget, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, listbox, tablelist, Ttk treeview, text, ctext

The `wcb::cbappend` Command

NAME

`wcb::cbappend` – Append to a callback list

SYNOPSIS

```
wcb::cbappend widgetName before|after option ?callback callback ...?
```

DESCRIPTION

This command is almost identical to `wcb::callback`. The only difference is that `wcb::cbappend` *appends* the arguments specified after the `option` parameter to the current callback list (if present), while `wcb::callback` *replaces* the old callbacks with these arguments.

KEYWORDS

callback, append, widget

The `wcb::cbprepend` Command

NAME

`wcb::cbprepend` – Prepend to a callback list

SYNOPSIS

```
wcb::cbprepend widgetName before|after option ?callback callback ...?
```

DESCRIPTION

This command is almost identical to `wcb::callback`. The only difference is that `wcb::cbprepend` *prepends* the arguments specified after the `option` parameter to the current callback list (if present), while `wcb::callback` *replaces* the old callbacks with these arguments.

KEYWORDS

callback, prepend, widget

The `wcb::cancel` Command

NAME

`wcb::cancel` – Cancel a widget command

SYNOPSIS

```
wcb::cancel ?script?
```

DESCRIPTION

This procedure is designed to be invoked from a before-callback for a widget command. It cancels the execution of that command and of the remaining callbacks, and evaluates the `script` argument in the global scope. If this argument is not specified, it defaults to the `bell` command.

The return value is the one obtained from `script` if this argument is specified and nonempty. Otherwise, the command returns an empty string.

KEYWORDS

cancel, command, widget

The `wcb::canceled` Command

NAME

`wcb::canceled` – Query the canceled status of a widget command

SYNOPSIS

```
wcb::canceled widgetName option
```

DESCRIPTION

Returns the value `1` if the most recent invocation of the widget operation corresponding to `widgetName` and `option` has been aborted by some before-callback by invoking `wcb::cancel`; otherwise, the return value is `0`. The arguments must fulfil the same restrictions as in the case of the `wcb::callback` command.

KEYWORDS

cancel, command, widget

The `wcb::extend` Command

NAME

`wcb::extend` – Extend the argument list of a widget command

SYNOPSIS

```
wcb::extend ?arg arg ...?
```

DESCRIPTION

This procedure is designed to be invoked from a before-callback for a widget command. It appends the values given in the optional `arg` parameters to the argument list of that command. The new argument list will be passed to the remaining callbacks for that command, too.

This procedure simply passes its parameters to the `lappend` command, called for the argument list of the respective widget operation.

KEYWORDS

extend, argument, command, widget

The `wcb::replace` Command

NAME

`wcb::replace` – Replace arguments of a widget command with new ones

SYNOPSIS

```
wcb::replace first last ?arg arg ...?
```

DESCRIPTION

This procedure is designed to be invoked from a before-callback for a widget command. It replaces the arguments having the indices `first` through `last` of that command with the

optional `arg` parameters. The new argument list will be passed to the remaining callbacks for that command, too. The arguments are numbered from `0` (see the table in the description of the `wcb::callback` command).

This procedure simply passes its parameters to the `lreplace` command, called for the argument list of the respective widget operation.

KEYWORDS

replace, argument, command, widget

The `wcb::pathname` Command

NAME

`wcb::pathname` – Query the path name of the widget corresponding to a Tcl command name

SYNOPSIS

```
wcb::pathname proxyCmd
```

DESCRIPTION

This procedure returns the path name of the widget corresponding to the Tcl command name `proxyCmd` passed to a callback.

When a before- or after-callback for a widget is invoked, the name of the proxy command associated with that widget is automatically appended to it as parameter. This procedure can be used within a callback to retrieve the path name of the widget from the command name passed to the callback as argument. It simply returns the string range obtained from `proxyCmd` by removing the `"::_"` prefix.

KEYWORDS

path name, command, widget

The `wcb::changeEntryText` Command

NAME

`wcb::changeEntryText` – Change the text of a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget

SYNOPSIS

```
wcb::changeEntryText widgetName string
```

DESCRIPTION

Replaces the text of the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` with `string`, by using the `delete` and `insert` operations. If the first subcommand is canceled by some before-`delete` callback then the procedure returns without inserting the new text; if the second operation is canceled by some before-`insert` callback then the command restores the original contents of the widget.

The procedure keeps the position of the insertion cursor. The return value is `1` on success and `0` on failure, i.e., if one of the attempted operations was canceled by some

before-callback.

KEYWORDS

Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

The `wcb::postInsertEntryLen` Command

NAME

`wcb::postInsertEntryLen` – Query the would-be length of the text in a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget after text insertion

SYNOPSIS

```
wcb::postInsertEntryLen widgetName string
```

DESCRIPTION

Returns the length of the text that would be contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` after inserting `string`.

KEYWORDS

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

The `wcb::postInsertEntryText` Command

NAME

`wcb::postInsertEntryText` – Query the would-be text of a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget after text insertion

SYNOPSIS

```
wcb::postInsertEntryText widgetName index string
```

DESCRIPTION

Returns the text that would be contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` after inserting `string` before the character indicated by `index`.

KEYWORDS

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

The `wcb::postDeleteEntryText` Command

NAME

`wcb::postDeleteEntryText` – Query the would-be text of a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget after text deletion

SYNOPSIS

```
wcb::postDeleteEntryText widgetName from ?to?
```

DESCRIPTION

Returns the text that would be contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` after deleting the characters starting with the one indicated by the index `from` and stopping just before `to`. If `to` is not specified then the

return value is the text that would be contained in the widget after deleting the single character given by `from`.

REMARK: This command has a variable number (2 or 3) of arguments, because the `delete` subcommand of the Tcl command associated with `widgetName` expects either one or two indices as arguments. For this reason, the correct way to invoke this command from within a before-`delete` callback is as shown in the following example:

```
proc myBeforeDeleteCallback {w args} {
    #
    # Get the text that would be contained in the widget after
    # deleting the characters specified by $args, which stands
    # for the one or two arguments passed to delete; pass these
    # arguments to wcb::postDeleteEntryText by expanding $args
    #
    set newText [eval [list wcb::postDeleteEntryText $w] $args]
    if {!some_condition_on_$newText} {
        wcb::cancel
    }
}
```

The following alternative, more elegant solution requires Tcl/Tk 8.5 or later:

```
proc myBeforeDeleteCallback {w args} {
    # . . .
    set newText [wcb::postDeleteEntryText $w {*}$args]
    . . .
}
```

KEYWORDS

delete, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Before-`insert` Callbacks for entry, spinbox, and combobox Widgets

NAME

`wcb::checkStrFor*`, `wcb::convStrTo*`, `wcb::checkEntryFor*`, `wcb::checkEntryLen` – Before-`insert` callbacks for Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, and Ttk combobox widgets

SYNOPSIS

```
wcb::checkStrForRegExp  exp w idx str

wcb::checkStrForAlpha       w idx str
wcb::checkStrForNum         w idx str
wcb::checkStrForAlnum       w idx str

wcb::convStrToUpper         w idx str
wcb::convStrToLower         w idx str
```

```
wcb::checkEntryForInt       w idx str
wcb::checkEntryForUInt  max w idx str
wcb::checkEntryForReal      w idx str
wcb::checkEntryForFixed cnt w idx str

wcb::checkEntryLen      len w idx str
```

DESCRIPTION

The `wcb::checkStrForRegExp` callback checks whether the string `str` to be inserted into the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` is matched by the regular expression `exp`; if not, it cancels the `insert` operation.

The three other `wcb::checkStrFor*` callbacks check whether the string `str` to be inserted into the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` is alphabetic, numeric, or alphanumeric, respectively; if not, they cancel the `insert` operation. These procedures just invoke the callback `wcb::checkStrForRegExp`, passing to it the Unicode-based patterns `{^[[:alpha:]]*$}`, `{^[[:digit:]]*$}`, and `{^[[:alnum:]]*$}`.

The `wcb::convStrTo*` callbacks replace the string `str` to be inserted into the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` with its uppercase or lowercase equivalent, respectively.

The `wcb::checkEntryFor*` callbacks check whether the text contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` after inserting the string `str` before the character indicated by the index `idx` would represent (the starting part of) an integer number, unsigned integer no greater than `max`, real number, or real number in fixed-point format with at most `cnt` digits after the decimal point, respectively; if not, they cancel the `insert` operation. `max` and `cnt` should be nonnegative numbers or `*`; `max = *`means: no upper bound for the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox value, while `cnt = *`stands for an unlimited number of digits after the decimal point.

The `wcb::checkEntryLen` callback checks whether the length of the text contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` after inserting the string `str` would be greater than `len`; if yes, it cancels the `insert` operation.

These callback procedures are implemented in the file `wcbEntry.tcl`, contained in the `scripts` directory. They return an empty string.

KEYWORDS

callback, insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Before-`insert` **Callbacks for text and ctext Widgets**

NAME

wcb::checkStrsFor*, wcb::convStrsTo* – Before-insert callbacks for text and ctext widgets

SYNOPSIS

```
wcb::checkStrsForRegExp exp w idx args

wcb::checkStrsForAlpha      w idx args
wcb::checkStrsForNum        w idx args
wcb::checkStrsForAlnum      w idx args

wcb::convStrsToUpper        w idx args
wcb::convStrsToLower        w idx args
```

DESCRIPTION

The wcb::checkStrsForRegExp callback checks whether the strings to be inserted into the text or ctext widget w, contained in the list args of the form string ?tagList string tagList ...?, are matched by the regular expression exp; if not, it cancels the insert operation.

The three other wcb::checkStrsFor* callbacks check whether the strings to be inserted into the text or ctext widget w, contained in the list args of the form string ?tagList string tagList ...?, are alphabetic, numeric, or alphanumeric, respectively; if not, they cancel the insert operation. These procedures just invoke the callback wcb::checkStrsForRegExp, passing to it the Unicode-based patterns {^[[:alpha:]\n]*$}, {^[[:digit:]\n]*$}, and {^[[:alnum:]\n]*$}.

The wcb::convStrsTo* callbacks replace the strings to be inserted into the text or ctext widget w, contained in the list args of the form string ?tagList string tagList ...?, with their uppercase or lowercase equivalents, respectively.

These callback procedures are implemented in the file wcbText.tcl, contained in the scripts directory. They return an empty string.

KEYWORDS

callback, insert, text, ctext, widget

License

These pages were converted from the original manual pages to Markdown (TIP 700 format). The text is unchanged; only markup without a place in the format (links without a target, layout images) may be lost. For parts under the Mozilla Public License 1.1 (tDOM) this is the description of modifications required by section 3.3.

This collection is not an official publication of the projects. Each part is under the terms of its original, which follow. The copyright notices of the original pages stay with each page.

Multi-entry widget package Mentry, version 4.5.1 Copyright (c) 1999-2026 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Index

I

index 3

M

mentry 3
mentryDateTime 28
mentryFixedPoint 33
mentryIPAddr 34
mentryIPv6Addr 36
mentryThemes 38
mentryWidget 42

W

wcbRef 52